

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI RAD br. 2304

**Vlastiti sustav za učenje
u visokoškolskom okruženju**

Luka Volarević

Zagreb, lipanj 2026.

ZAVRŠNI ZADATAK br. 2304

Pristupnik: **Luka Volarević (0036561873)**
Studij: Elektrotehnika i informacijska tehnologija i Računarstvo
Modul: Računarstvo
Mentor: prof. dr. sc. Igor Mekterović

Zadatak: **Vlastiti sustav za učenje u visokoškolskom okruženju**

Opis zadatka:

Potrebno je razviti i evaluirati web-aplikaciju za učenje u visokoškolskom okruženju, koja studentima omogućuje strukturirano i adaptivno savladavanje gradiva pojedinog kolegija na temelju zadanih nastavnih materijala. Aplikacija omogućuje učitavanje i integraciju različitih vrsta obrazovnih sadržaja vezanih uz jedan kolegij, uključujući predavanja, laboratorijske vježbe, ispitne rokove, upute, primjere zadataka i ostale relevantne informacije. Na temelju prikupljenih materijala sustav provodi poluautomatsku analizu i strukturiranje sadržaja pomoću velikih jezičnih modela te izgrađuje plan učenja. Sustav studenta vodi kroz gradivo u logičkom i vremenskom slijedu, pri čemu se redovito generiraju provjere znanja u obliku kratkih testova ili zadataka. Rezultati tih provjera koriste se za procjenu napretka te eventualnu prilagodbu daljnjeg redoslijeda gradiva (ponavljanje gradiva, dodatni testovi i sl.). Time se ostvaruje nadzirani mehanizam učenja koji kombinira planiranje, praćenje i evaluaciju znanja. U sklopu rada potrebno je analizirati slične postojeće pristupe sustavima za učenje, osmisliti arhitekturu sustava koja podržava učitavanje, obradu i strukturiranje heterogenih obrazovnih materijala te na koncu implementirati web-aplikaciju s podrškom za generiranje vlastitih, lokalnih planova učenja i provjera znanja. Sustav je potrebno evaluirati na primjeru jednog ili više kolegija. Na kraju rada donijeti kritički osvrt na ostvareno i dati smjernice za budući razvoj.

Rok za predaju rada: 12. lipnja 2026.

*Velika zahvala mojoj curi Danijeli Šimić
Također zahvaljujem svom mentoru prof. dr. sc. Igoru Mekteroviću*

Sažetak

Vlastiti sustav za učenje u visokoškolskom okruženju

Digitalizacija je u suvremenom visokoškolskom okruženju unijela znatne promjene. Jedna od tih promjena je olakšan pristup materijalima kolegija. Studenti danas istovremeno barataju prezentacijama, skriptama, laboratorijskim vježbama i prošlogodišnjim ispitima iz raznih kolegija. Iako ovime imaju pristupačniju mogućnost učenja, heterogenost izvora materijala znatno otežava kontinuirano učenje i praćenje napretka kroz semestar. Posljedično, dolazi do otežanog planiranja učenja i nepravilne raspodjele vremena. Zbog toga se sve više pojavljuju alati za oblikovanje nastavnih materijala u definirane cjeline. U okviru ovog rada opisano je rješenje **RoadmapAI**, aplikacija koja koristi velike jezične modele (eng. *Large Language Models*) za preoblikovanje heterogenih nastavnih materijala u strukturirane, personalizirane i vođene planove učenja. Ovo rješenje studentima olakšava samoorganizaciju polaganja semestra te uvodi interaktivni pregled napretka u kolegijima.

Ključne riječi: plan učenja; nastavni materijali; LLM; ekstrakcija teksta; asinkrona mikrousluga

Summary

Dedicated Learning System in a Higher Education Environment

Digitalization has brought significant changes to the contemporary higher education environment. One of these changes is eased access to course materials. Today, students simultaneously handle presentations, scripts, laboratory exercises, and exams from previous years across various courses. Although this provides them with a more accessible learning opportunity, the heterogeneity of material sources considerably hinders continuous learning and tracking progress throughout the semester. Consequently, this leads to difficulties in study planning and improper time distribution. Because of this, tools for structuring teaching materials into defined units are increasingly emerging. Within the framework of this paper, the RoadmapAI solution is described, an application that utilizes Large Language Models to reshape heterogeneous teaching materials into structured, personalized, and guided learning plans. This solution facilitates students' self-organization in passing the semester and introduces an interactive overview of progress in their courses.

Keywords: study roadmap; course materials; LLM; text extraction; async microservice

Sadržaj

1. Uvod	8
1.1. Motivacija	8
1.2. Cilj rada	8
1.3. Pregled strukture rada	9
2. Analiza sličnih rješenja sustava za učenje	10
2.1. Microsoft Learn	10
2.2. Moodle	11
2.3. Udemy	12
2.4. Napredne AI-integracije	12
2.5. Usporedna ocjena i implikacije za RoadmapAI	13
3. Arhitektura i dizajn sustava RoadmapAI	14
3.1. Funkcionalni i nefunkcionalni zahtjevi	14
3.2. Arhitektura baze podataka	15
3.3. Enumeracije korištene u modelu	18
3.4. Indeksi i ograničenja	18
3.5. Autentikacija i sigurnost sustava	19
4. Tehnologije i razvojno okruženje	21
4.1. Korisničko sučelje	21
4.2. Poslužiteljski dio	22
4.2.1. Vanjske ovisnosti	23
4.3. Integracija umjetne inteligencije	23
4.4. Infrastruktura baze podataka	24
4.5. Infrastruktura u oblaku, kontinuirana integracija i kontinuirana isporuka	25
5. Implementacija aplikacije i pregled funkcionalnosti	27
5.1. Upravljanje materijalima i kolegijima	28
5.2. Interaktivno savladavanje lekcija	30
5.3. Evaluacija znanja kroz ispite i prilagodive povratne informacije	33
5.4. Parsiranje odgovora	35
5.5. Oblikovanje upita za velike jezične modele	36
5.5.1. Upit za plan učenja	37
5.5.2. Upit za lekcije	38
5.5.3. Upit za ispite	39
5.5.4. Upit za povratnu informaciju	40
5.5.5. Primjer toka generiranja	41

6. Zaključak	44
6.1. Osvrt na ostvareni projekt	44
6.2. Mogućnosti daljnjeg razvoja aplikacije	44
Literatura	46

1. Uvod

1.1. Motivacija

Glavna motivacija za rješenje je smanjenje opterećenja studenata pri organizaciji učenja iz heterogenih izvora tijekom semestra. Postoje razne komercijalne edukacijske platforme koje na različite načine pristupaju tom problemu, no rijetko koja omogućuje potpunu personalizaciju uz očuvanje strukture kolegija. RoadmapAI pokušava ujediniti prednosti poznatih platformi poput Microsoft Learn, Udemy i Moodlea, koje u kombinaciji s umjetnom inteligencijom imaju potencijal maksimizirati **produktivnost** studenata.

Također, jedan od ciljeva je smanjiti vrijeme koje student treba potrošiti na samoorganizaciju kolegija te preusmjeriti to vrijeme na učenje i duboko razumijevanje gradiva. Time se značajno povećava produktivnost i konzistentnost studenata kroz semestre, unatoč potencijalnom neiskustvu u snalaženju u zahtjevnim semestrima s velikim brojem kolegija.

Još jedna motivacija za rješenje je pomoć profesorima u organizaciji nastave. RoadmapAI može poslužiti profesorima kao alat ili vodič za strukturiranje kolegija iz tjedna u tjedan, do razine utjecaja i detalja koju sami žele definirati. [1], [2]

1.2. Cilj rada

Cilj ovog rada je razvoj web aplikacije RoadmapAI. Aplikacija omogućuje uvoz obrazovnih materijala te njihovu automatiziranu obradu s pomoću velikih jezičnih modela kako bi se izgradio strukturiran **plan za učenje**.

Aplikacija treba pružiti mogućnost korisniku da učitava materijale relevantne za kolegij te da na osnovi tih materijala dobije plan učenja koji se sastoji od lekcija, modula i ispita. Radi točnosti materijala, primarni izvor informacija trebaju biti sami materijali, dok model služi za povezivanje gradiva u prikladan tok. Aplikacija treba biti jednostavna i intuitivna za korištenje.

Još jedan cilj rada je ispitati i zaključiti do koje mjere umjetna inteligencija može pomoći studentima u organizaciji u semestru, kao i samim profesorima u sastavljanju nastave.

Analizom implementiranog rješenja na kraju će biti donesen zaključak o uspješnosti implementacije umjetne inteligencije u procesu obrazovanja studenata.

1.3. Pregled strukture rada

Rad je organiziran u šest poglavlja.

Prvo poglavlje služi kao kratki uvod u rad te sažetak problema i motivacije rješenja.

U drugom poglavlju provodi se analiza postojećih edukacijskih sustava te se prikazuju načini na koje napredne AI tehnologije mogu unaprijediti obrazovanje.

Treće poglavlje opisuje arhitekturu sustava RoadmapAI, funkcionalne i nefunkcionalne zahtjeve, dizajn baze podataka te implementaciju sigurne autentikacije.

Četvrto poglavlje obrađuje tehničku stranu aplikacije, točnije odabrane tehnologije, programske jezike i radne okvire, kao i odabir baze podataka i infrastrukture u oblaku.

Peto poglavlje fokusira se na implementaciju ključnih funkcionalnosti poput upravljanja planom učenja, asinkronog generiranja sadržaja, generiranja interaktivnih lekcija i ispita te evaluacije znanja studenta na temelju rezultata.

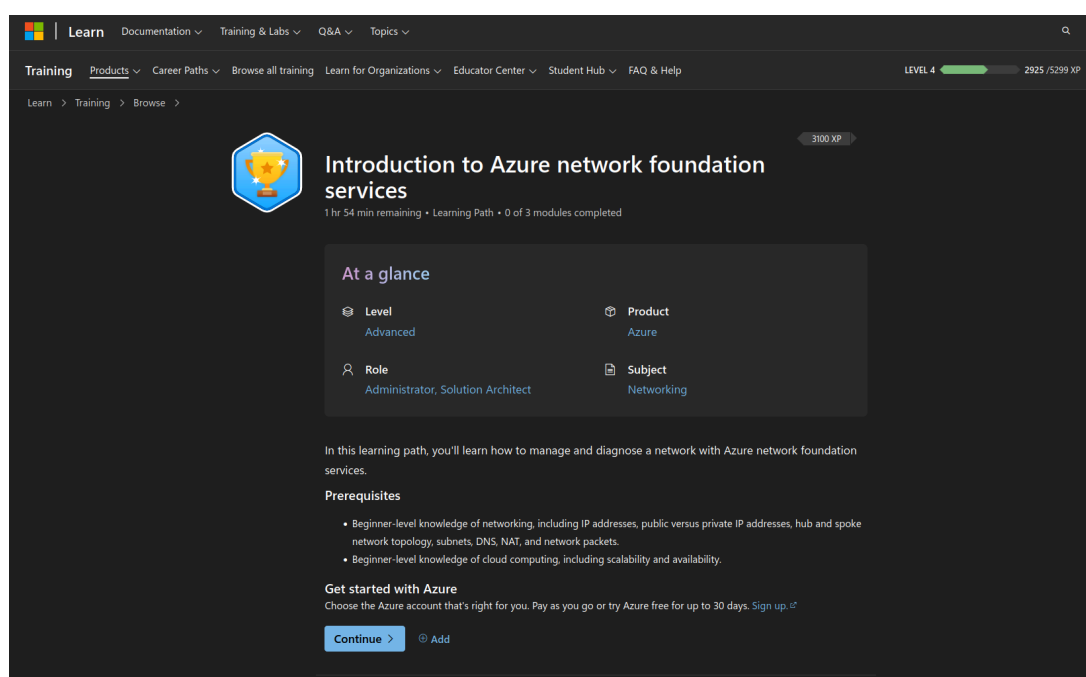
U šestom i posljednjem poglavlju se donosi zaključak rada, analiza ispunjenja postavljenih ciljeva i prijedloge za buduća poboljšanja.

2. Analiza sličnih rješenja sustava za učenje

U ovom poglavlju analizira se skup postojećih rješenja u području digitalnog obrazovanja. Cilj analize je prepoznati ključna obilježja postojećih platformi, njihove pristupe strukturiranju sadržaja i načine evaluacije znanja te izvesti zaključke koji će poslužiti kao smjernice za dizajn RoadmapAI rješenja.

Izdvojene platforme su Microsoft Learn, Udemy i Moodle, a motivacija za analizu ovih platformi je njihov specifičan pristup **digitalnoj edukaciji**. [1], [2], [3]

2.1. Microsoft Learn



Slika 1: Microsoft Learn - primjer sučelja

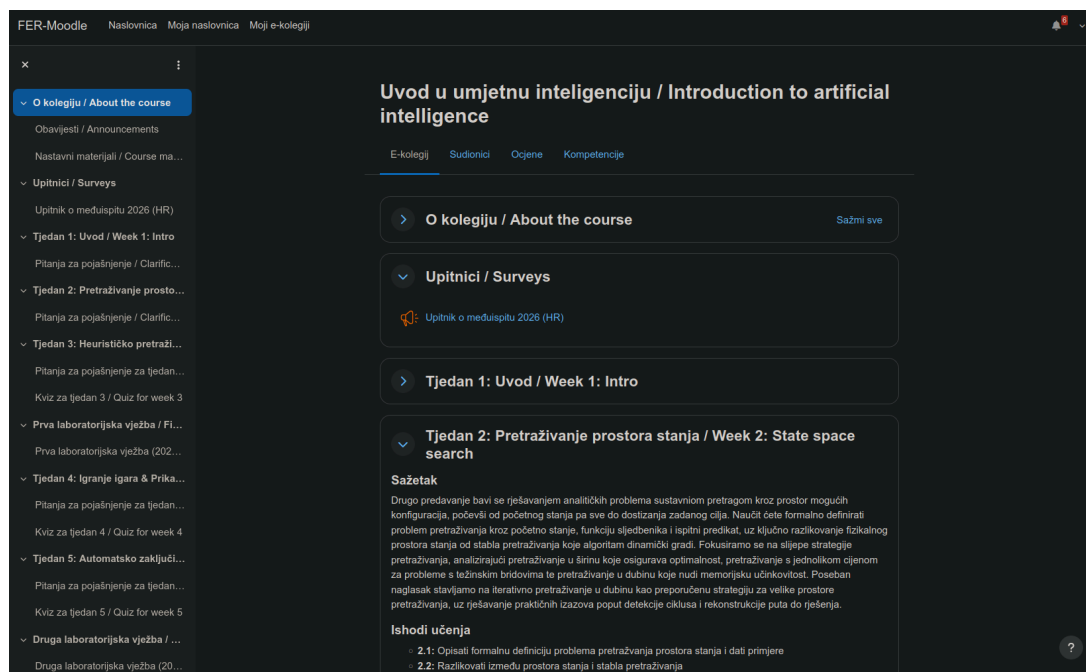
Microsoft Learn je primjer **komercijalne** platforme koja nudi u potpunosti sastavljene planove učenja. Gradivo je usmjereno podučavanjem korištenja Microsoft usluga. Na platformi se gradivo organizira u strukturu put (eng. *path*), modul (eng. *module*) i lekcija (eng. *lesson*). Svaka lekcija završava sa kratkom provjerom znanja koja označava položenu lekciju i mogućnost napretka. Prednosti opisanog pristupa su jasna definiranost ishoda učenja, konkretno vođenje kroz učenje i kratke provjere koje pomažu korisniku usvojiti gradivo. [1]

Microsoft Learn slijedi pristup čiji je sadržaj autor unaprijed složio, što znači da svi korisnici prate iste korake u planu učenja. Takav pristup pospješuje standardizaciju is-

hoda te lako i kontrolirano praćenje napretka. Nedostatak je ograničenost fleksibilnosti i personaliziranosti načina savladavanja gradiva, što varira od korisnika do korisnika.

Također, Microsoft Learn pruža pregled napretka kroz različite putove i module, čime se korisnik dodatno motivira sustavom nagrada u obliku bodova i razina.

2.2. Moodle

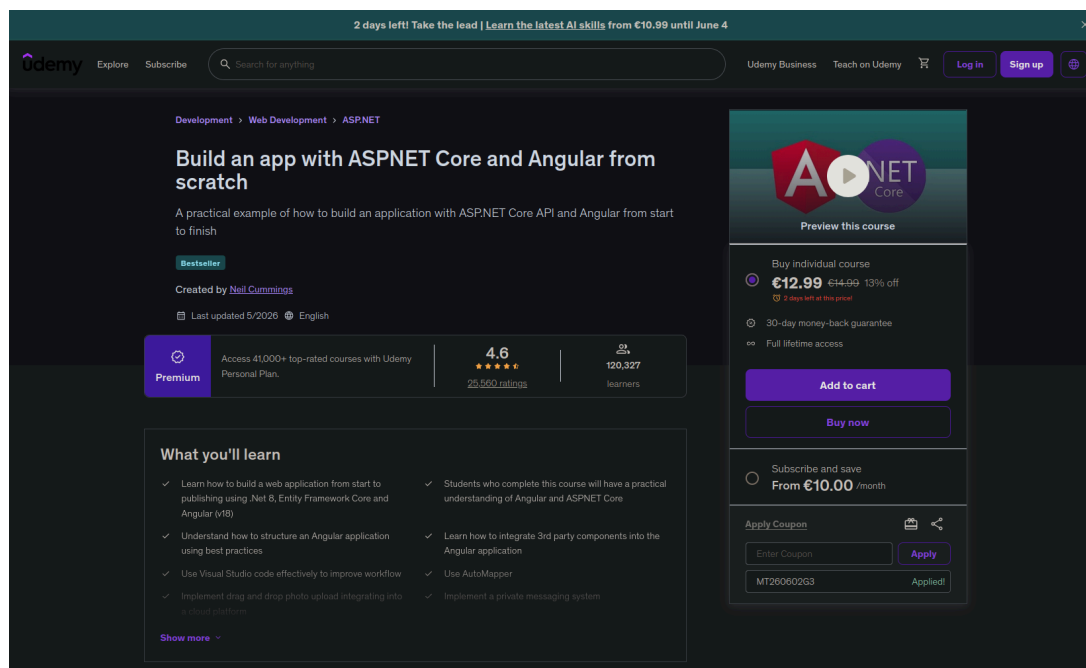


Slika 2: Moodle - primjer stranice kolegija

Moodle je platforma koja je sve više korištena u obrazovnom sustavu. Unutar aplikacije, visokoškolske institucije i predavači mogu jednostavno objavljivati različite formate dokumenata, poput skripti, prezentacija i zadataka za vježbu, uz opcije postavljanja zadataka ili pokretanja foruma. Iako platforma nudi mehanizme za administraciju sadržaja i praćenje aktivnosti studenata, ona ne posjeduje opcije za automatsko strukturiranje ili inteligentnu organizaciju materijala. [3]

Zbog takvog slobodnog modela dodavanja datoteka, Moodle ostvaruje ulogu repozitorija heterogenih izvora. To s jedne strane pruža veliku slobodu pri oblikovanju samog kolegija, ali s druge strane često dovodi do nejednakosti u organizaciji gradiva između različitih predmeta. Upravo ta neusklađenost otežava pokušaje standardizacije izvora učenja u semestru i onemogućuju automatsko generiranje prilagođenih planova učenja za svakog studenta.

2.3. Udemy



Slika 3: Udemy - struktura tečaja

Udemy je rješenje koje koristi tržišni model. Korisnici objavljuju planove učenja s vlastito odabranim modulima i lekcijama te se distribuira na monetizirani način. Gradivo je u pravilu objašnjeno video-lekcijama i tekstualnim skriptama. [2]

Slično kao i na platformi Microsoft Learn, Udemy sadrži unaprijed izrađene planove učenja kako bi korisničko iskustvo bilo ujednačeno. Glavna razlika je u tome što na Udemy platformi kvaliteta i stilovi nastavnog sadržaja dosta variraju, što je izravna posljedica otvorenog modela objave materijala. Prednost je što korisnik lakše može pronaći predavača koji odgovara njegovom stilu učenja, no s druge strane, akademska točnost i provjerenost tih materijala nisu uvijek zajamčene.

2.4. Napredne AI-integracije

U navedenim platformama sve se češće pojavljuju integracije naprednih LLM modela u obrazovne tokove (npr. asistenti za objašnjavanje odgovora te personalizirane preporuke sljedećih sadržaja). Takvi pristupi omogućuju personalizaciju puta učenja i dodatnu podršku korisnicima.

Također, sve učestaliji primjer je AI asistent integriran u samu platformu, koji korisnicima pomaže pri korištenju sustava te odgovara na često postavljena pitanja.

2.5. Usporedna ocjena i implikacije za RoadmapAI

Analiza platformi digitalne edukacije pokazuje tri dominantna obrasca:

1. striktno **strukturirani** edukacijski putevi
2. **fleksibilni** repozitoriji sadržaja s velikom varijabilnošću
3. slobodni model objavljivanja materijala iz raznih izvora koji povećava **personaliziranost**

Za RoadmapAI važno je spojiti prednosti ovih pristupa: automatsko strukturiranje sadržaja u slijedne module i lekcije (kao kod komercijalnih platformi), zadržati fleksibilnost u radu s heterogenim izvorima (kao kod fleksibilnih repozitorija) te omogućiti selektivnu personalizaciju putem jezičnih modela.

Zaključno, postojeća rješenja pružaju vrijedne dizajnerske smjernice, ali ne zadovoljavaju u potpunosti potrebu za istovremenim generiranjem strukturiranih, fleksibilnih i personaliziranih planova učenja. Upravo tu RoadmapAI nudi svoje rješenje.

3. Arhitektura i dizajn sustava RoadmapAI

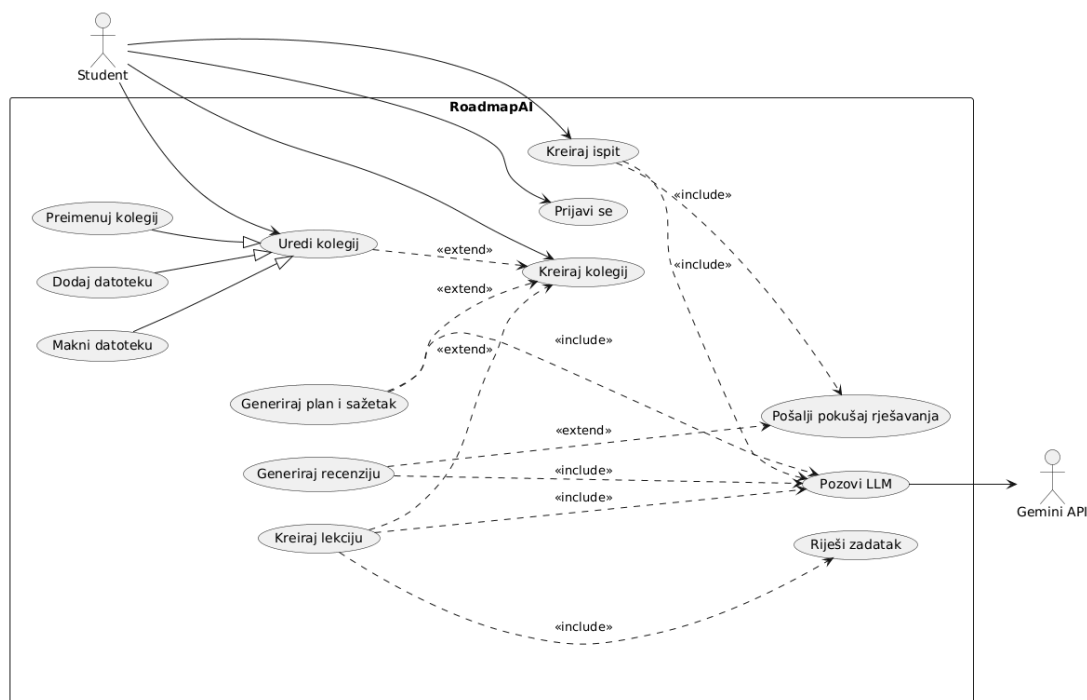
3.1. Funkcionalni i nefunkcionalni zahtjevi

U nastavku su navedeni funkcionalni zahtjevi koji definiraju ključne mogućnosti sustava:

1. Sustav mora omogućiti uvoz datoteka u formatima PDF, DOCX, PPTX, MD i TXT, njihovu tekstualnu ekstrakciju i perzistentno spremanje.
2. Sustav mora omogućiti korisniku stvaranje kolegija na osnovu uvezenih nastavnih materijala te generiranje sažetka i strukturiranog plana učenja podijeljenog na module, lekcije i ispite.
3. Sustav mora omogućiti dodavanje i uklanjanje materijala u postojećem planu učenja.
4. Rezultati pokušaja rješavanja ispita trebaju biti pohranjeni i na osnovu njih korisnik mora moći dobiti povratnu informaciju o svom napretku kroz kolegij.
5. Sustav mora omogućiti pregled napretka korisnika kroz kolegij na razini modula, lekcija i ispita.
6. Sustav mora omogućiti korisniku neograničeno ponovno generiranje plana učenja i lekcija.
7. Sustav mora omogućiti pregled povijesti riješenih ispita i ostvarenih rezultata.
8. Sustav mora omogućiti pristup ispitu tek nakon što korisnik odradi sve lekcije unutar modula, te pristup završnom ispitu nakon rješavanja cijelog plana učenja kolegija.

Nefunkcionalni zahtjevi opisuju kvalitativne značajke sustava, tehnička ograničenja te standarde koje sustav mora zadovoljiti:

1. Sustav treba pružati skalabilnost implementirajući odvojeni API sloj od AI mikros-luge.
2. Sustav treba pružati mogućnost oporavka od greške vanjskog LLM modela ponovnim pokušajem.
3. Korisnik u svakom koraku treba biti informiran kroz status ingestije materijala i generiranja sadržaja.
4. Pružanje autorizacije i autentikacije korisnika koristeći ASP.NET Core Identity i HttpOnly Secure cookie za JWT token.
5. Sustav mora omogućiti odgođeno generiranje sadržaja za pojedinačne lekcije i ispite na zahtjev korisnika.
6. Sustav mora omogućiti generiranje lekcija i ispita kojima su priloženi materijali primarni izvor istine (eng. *source of truth*)
7. Korisničko sučelje mora odgovarati WCAG AA standardima.



Slika 6: UML dijagram studenta i LLM API-ja

3.2. Arhitektura baze podataka

Podatkovni sloj implementiran je putem `AppDbContext` klase koja predstavlja centralni Entity Framework Core kontekst za pristup bazi podataka. `AppDbContext` definira skup `DbSet` entiteta koji odgovaraju tablicama u bazi te upravlja mapiranjem između C# modela i relacijske strukture baze, kao i izvođenjem CRUD operacija nad podacima.

`Course` predstavlja osnovnu jedinicu sustava, odnosno kolegij koji korisnik stvara i uređuje. Uz identifikator i vezu na vlasnika (`UserId`), u tablici se čuvaju naziv, sažetak, status te vremenske oznake stvaranja i izmjene. Polje `Status` koristi se za praćenje faze u kojoj se kolegij nalazi, pa frontend prema tome može prikazati je li kolegij još u obradi, spreman za generiranje plana učenja ili već dovršen.

`CourseMaterial` opisuje svaku učitane datoteku. Ovdje se čuva naziv datoteke, tip sadržaja, veličina i putanja prema spremištu. Dodatna polja `Type` i `Status` opisuju vrstu materijala i stanje obrade, dok `ExtractedText` sadrži tekst koji je izvučen tijekom ingestije. Upravo taj zapis omogućuje rad nad sadržajem, a ne nad samom binarnom datotekom.

`CourseRoadmap` predstavlja rezultat generiranja strukture kolegija. U ovoj tablici čuva se markdown sažetak plana učenja i vrijeme njegove izrade. Veza na `CourseId` je jedins-

tvena, što znači da svaki kolegij ima jedan roadmap kao centralnu strukturu iz koje se dalje grade moduli i lekcije.

RoadmapModule grupira lekcije u logične cjeline. OrderIndex definira redoslijed prikaza, a Title naziv modula. Polje IsCompleted služi za označavanje završetka modula u korisničkom sučelju. Modul je tako više od običnog spremnika za lekcije. On predstavlja logičku i vizualnu razinu kroz koju se korisnik kreće kroz gradivo.

RoadmapLesson predstavlja pojedinačnu lekciju. Uz osnovni naslov i markdown sadržaj, ovdje se nalaze i polja za mali zadatak na kraju lekcije: pitanje, četiri ponuđena odgovora, točna opcija i pomoćni tekst. Status lekcije pokazuje je li ona još negenerirana, u obradi, generirana, u stanju pogreške ili dovršena. Tako lekcija ne služi samo za prikaz sadržaja, nego i za praćenje ciklusa generiranja.

RoadmapExam i RoadmapExamQuestion čine ispitni dio modela. Ispit sadrži naslov, oznaku je li riječ o završnom ispitu te status koji pokazuje je li zaključan, spreman ili dovršen. Pitanja ispita sadrže tekst pitanja, ponuđene odgovore, točnu opciju i kratku pomoć koja se pokaže ako je prvi pokušaj rješavanja neuspješan.

RoadmapExamAttempt i RoadmapExamAttemptAnswer služe za bilježenje rada korisnika na ispitima. U pokušaju se čuva vrijeme početka, vrijeme predaje, postotak uspješnosti i oznaka prolaza, dok se na razini odgovora pohranjuje tekst pitanja, točna i odabrana opcija te informacija je li odgovor bio točan. Time se može rekonstruirati cijeli ispitni pokušaj i iz njega izvući povratne informacije.

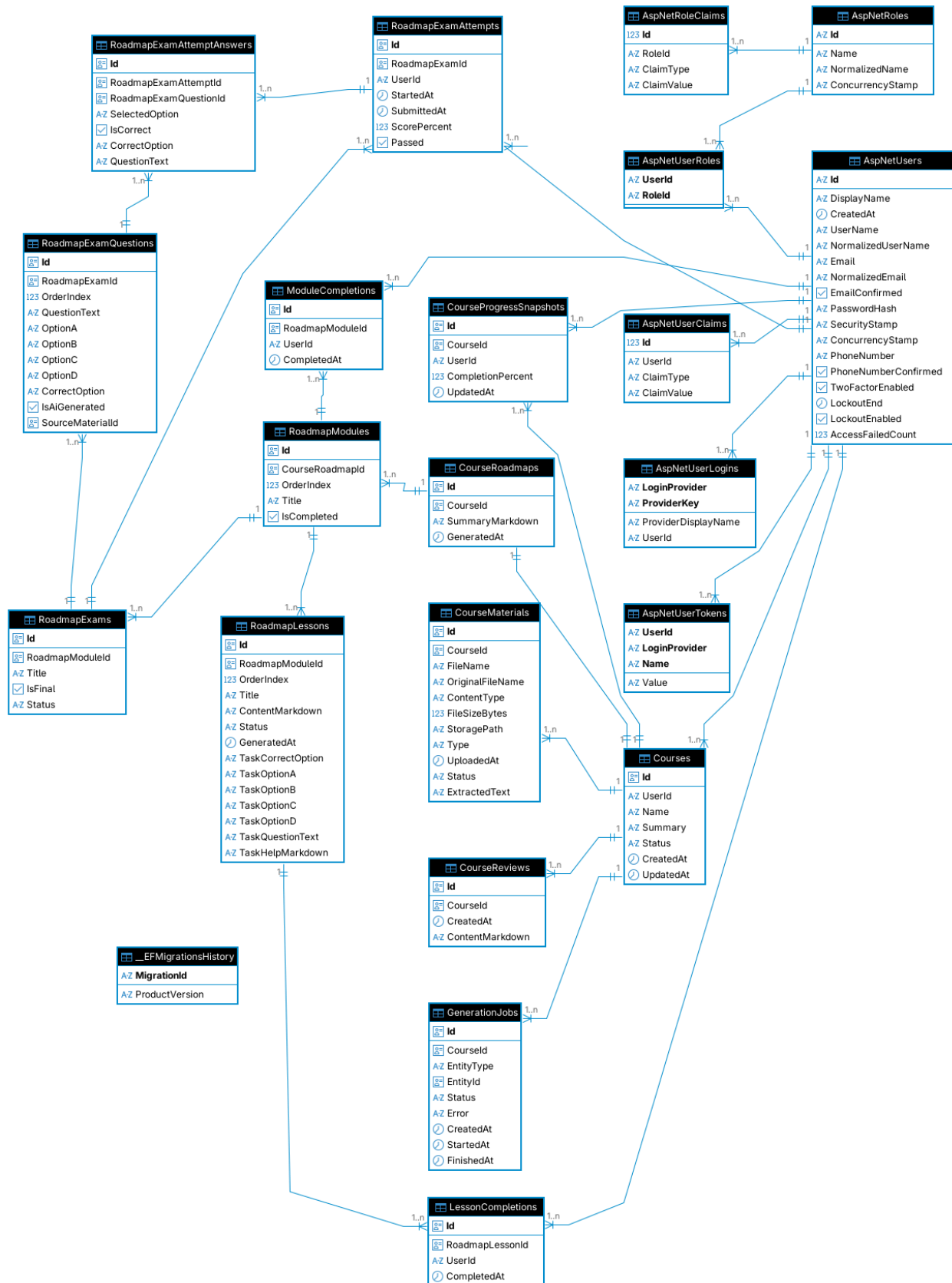
CourseReview je jednostavniji entitet za tekstualne povratne informacije o kolegiju. Sadrži datum i markdown sadržaj te služi za komentare ili sažetke koje sustav može kasnije prikazati korisniku.

LessonCompletion i ModuleCompletion bilježe dovršavanje lekcija i modula po korisniku. Ove tablice su važne jer se na temelju njih izračunava napredak i otključavanje sljedećih dijelova plana učenja.

CourseProgressSnapshot služi kao predmemorija izračunatog napretka po kolegiju i korisniku. Umjesto da se svaki put ponovno računa napredak iz više tablica, ova tablica čuva već izračunat postotak i vrijeme zadnje izmjene. Time se ubrzava prikaz dashboarda i pregled napretka.

GenerationJob je zapis za generacijske operacije. Svaki posao sadrži vrstu entiteta, identifikator ciljanog entiteta, status, opcionalnu poruku greške te vremenske oznake početka i završetka.

AppUser proširuje ASP.NET Core Identity korisnika dodatnim poljima DisplayName i CreatedAt.



Slika 7: Arhitektura baze

3.3. Enumeracije korištene u modelu

Enumeracije se koriste kroz cijelu aplikaciju. Primarni razlog njihove uporabe je obavljanje korisnika o trenutnom statusu tijekom cijelog procesa, od procesiranja ulaznih datoteka do generiranja lekcija i ispita. Entity Framework Core omogućuje izravno korištenje enumeracija kao tipova podataka, čime se postiže DRY princip (eng. *Don't Repeat Yourself*) i smanjuje mogućnost pojave nekonzistentnih vrijednosti.

Korištene enumeracije su:

- `CourseStatus`: `Processing`, `Ready`, `RoadmapGenerating`, `RoadmapReady`, `Error`, `Completed`.
- `MaterialStatus`: `Uploaded`, `Ingesting`, `Ready`, `Error`.
- `MaterialType`: `Lecture`, `Exam`, `LabExercise`, `Other`.
- `LessonStatus`: `NotGenerated`, `Generating`, `Generated`, `Error`, `Completed`.
- `ExamStatus`: `Locked`, `Ready`, `Completed`.
- `GenerationJobStatus`: `Queued`, `Running`, `Done`, `Failed`.
- `GenerationJobEntityType`: `CourseRoadmap`, `Lesson`, `Exam`.

3.4. Indeksi i ograničenja

U bazi podataka implementiran je niz **indeksa** koji ubrzavaju najčešće korištene upite te osiguravaju konzistentnost i jedinstvenost podataka unutar tablica. `Course` ima indeks na `UserId`, što olakšava dohvat svih kolegija jednog korisnika. `CourseMaterial` ima indeks na kombinaciju `CourseId` i `Status`, jer se materijali najčešće čitaju po kolegiju i stanju obrade. `CourseReview` ima indeks na `CourseId` i `CreatedAt` kako bi se recenzije mogle sortirati i filtrirati po vremenu.

Na roadmap razini `CourseRoadmap` ima jedinstveni indeks na `CourseId`, jer kolegij može imati samo jedan aktivni roadmap. `RoadmapModule` i `RoadmapLesson` imaju jedinstvene kombinirane indekse na parove `CourseRoadmapId/OrderIndex` i `RoadmapModuleId/OrderIndex`, što osigurava stabilan redoslijed prikaza modula i lekcija. `RoadmapLesson` dodatno ima indeks na `RoadmapModuleId` i `Status` jer se često filtrira po stanju generiranja.

`RoadmapExamQuestion` ima jedinstveni indeks na `RoadmapExamId` i `OrderIndex`, a polje `CorrectOption` je ograničeno na jednu znamenku ili znak. `RoadmapExamAttempt` ima indeks na `RoadmapExamId`, `UserId` i `SubmittedAt`, što pomaže pri dohvatima pokušaja po korisniku i vremenu. `LessonCompletion`, `ModuleCompletion` i `CourseProgressSnapshot` svi imaju jedinstvene indekse na parovima koji povezuju sadržaj i korisnika, čime se sprječava duplikacija zapisa za isti napredak. `GenerationJob` ima indeks na `CourseId`, `Status` i `CreatedAt` kako bi se brzo dohvatili aktivni ili nedavno pokrenuti poslovi.

3.5. Autentikacija i sigurnost sustava

Autentikacijski i autorizacijski tok implementiran je kroz nekoliko dijelova aplikacije: `AuthController`, JWT konfiguraciju u datoteci `Program.cs` (ulaznoj točki API dijela aplikacije) te middleware cjevovod koji se registrira pozivima `app.UseAuthentication();` i `app.UseAuthorization();`.

Identitet korisnika vodi se kroz ASP.NET Core Identity, no koristi se vlastita implementacija klase `AppUser` nasljeđivanjem Identity korisnika kako bi se mogla definirati dodatna svojstva i ponašanja potrebna za aplikaciju.

Prilikom prijave i registracije API generira JWT token, pri čemu token sadrži metapodatke poput korisničkog imena i e-mail adrese. Token je potpisan simetričnim ključem `Jwt:Secret`, koji se iz sigurnosnih razloga pohranjuje u varijablama okruženja. Time je ostvarena kontrola nad vijekom trajanja tokena bez dodatne složenosti na klijentskoj strani. Vijek trajanja tokena konfigurira se putem varijable okruženja `Jwt:ExpiryDays`, a standardno iznosi 7 dana. Nakon isteka tokena, korisnik ponovnom prijavom prima svoj novi valjani JWT token.

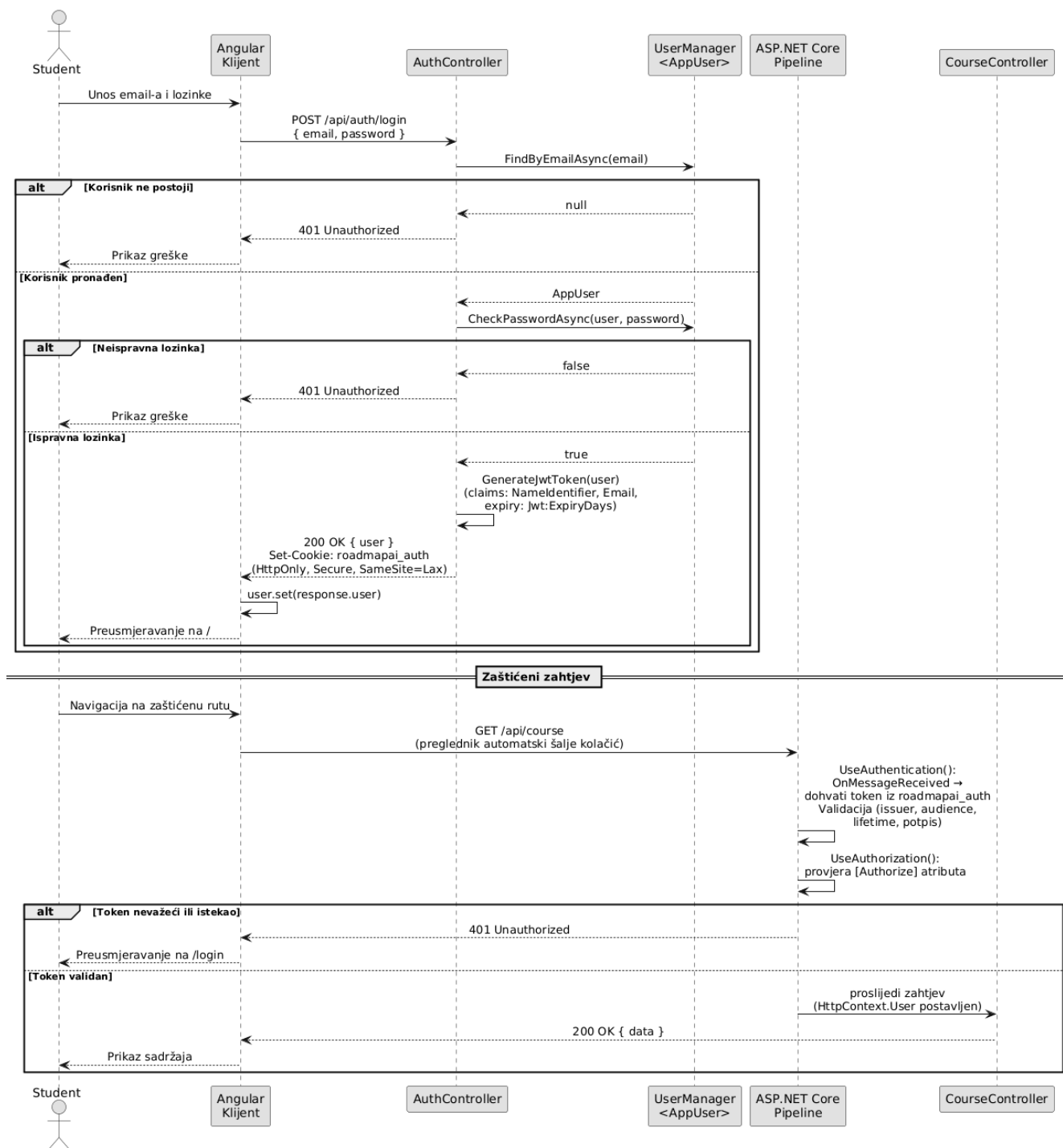
Token se sprema u obliku `HttpOnly` kolačića `roadmapai_auth` uz postavke `HttpOnly = true`, `Secure = isHttps` i `SameSite = Lax`. Takav pristup smanjuje rizik od krađe tokena putem XSS napada te bolje odgovara web aplikacijama koje se oslanjaju na pregledničke kolačiće. [4]

CORS politika `AllowAngular` dopušta zahtjeve iz klijentske aplikacije, dok `AllowCredentials()` omogućuje slanje kolačića u HTTP zahtjevima kada je to potrebno. Na klijentskoj strani implementiran je presretač (eng. *interceptor*) koji automatski postavlja opciju `withCredentials` za zahtjeve prema API-ju, čime preglednik uz zahtjev šalje autentikacijske kolačiće. Na poslužiteljskoj strani JWT middleware dohvaća token iz kolačića i provodi njegovu validaciju.

Tijek prijave i autorizacije:

1. Klijent poziva `POST /api/auth/login` s korisničkim vjerodajnicama.
2. Backend provjerava korisnika putem `UserManager` servisa te u slučaju uspješne autentikacije generira JWT token.
3. JWT token vraća se klijentu kroz `HttpOnly` kolačić `roadmapai_auth`.
4. Na svim budućim zahtjevima preglednik automatski šalje kolačić prema API-ju, dok middleware `JwtBearer` validira token i po potrebi ga dohvaća putem mehanizma `OnMessageReceived`.

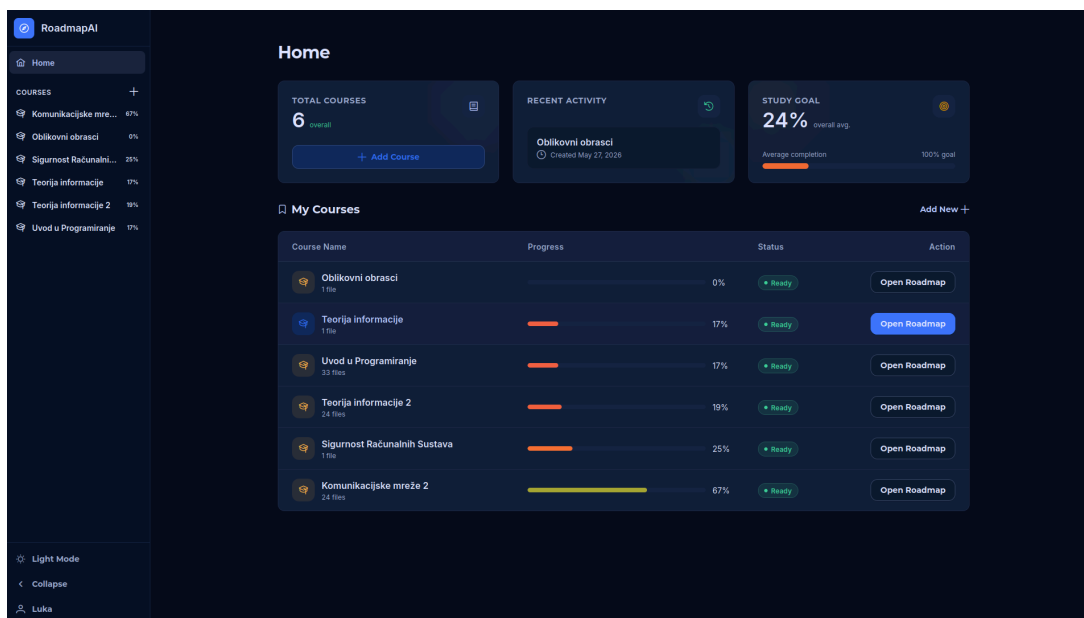
5. UseAuthentication() i UseAuthorization() osiguravaju da krajnje točke označene atributom [Authorize] zahtijevaju valjan autentikacijski token.



Slika 8: Sekvencijski dijagram JWT prijave

4. Tehnologije i razvojno okruženje

4.1. Korisničko sučelje



Slika 4: Primjer korisničkog sučelja (UI)

U nastavku su opisani detalji odabira tehnologija za izradu **korisničkog sučelja** (eng. *frontend*). Korištene tehnologije predstavljaju industrijski standard te su dokazano pouzdane kroz dugogodišnju primjenu u produkcijskim sustavima diljem svijeta. Za razvoj korisničkog sučelja korišteni su Angular, Tailwind CSS i PrimeNG knjižnica komponenti.

Angular je odabran zbog svoje snažne podrške za tipizirani razvoj, a njegov bogati ekosustav i jednostavna implementacija obrazaca čine ga pogodnim za efikasnu izgradnju sučelja. Kod je pisan u striktno tipiziranom TypeScript jeziku, što poboljšava sigurnost i kvalitetu tijekom izvođenja aplikacije. Dijelovi sučelja organizirani su u komponente, a diljem aplikacije koriste se signali (eng. *signals*) za upravljanje stanjem. Rute (eng. *routes*) konfigurirane su za odgođeno učitavanje (eng. *lazy loading*) kako bi se smanjilo početno vrijeme učitavanja stranica. [5]

Za uređivanje stila korisničkog sučelja korišten je Tailwind CSS, što je omogućilo brz razvoj dizajna kroz uslužne CSS klase (eng. *utility CSS classes*) i jednostavno održavanje vizualne konzistentnosti kroz sve stranice. Za složenije i interaktivne elemente korišten je PrimeNG, jedna od najpoznatijih knjižnica komponenti u Angular ekosustavu. Forme su implementirane korištenjem reaktivnog pristupa (eng. *reactive forms*), uz definirana validacijska pravila na strani klijenta i poslužitelja. Boje korištene u aplikaciji

definirane su kroz Tailwind konfiguraciju kako bi se izbjeglo hardkodiranje vrijednosti, pri čemu je za svaku boju definirana odgovarajuća varijanta za svijetlu i tamnu temu. [6], [7]

Korisničko sučelje razvijeno je u skladu s WCAG AA smjernicama, uz naglasak na čitljivost i korištenje semantičkih ARIA oznaka, čime je povećana pristupačnost aplikacije. [8]

4.2. Poslužiteljski dio

Poslužiteljski dio sustava (eng. *backend*) funkcionira na REST principu. Korištene su tehnologije bazirane na jeziku C#, a to su .NET 10 i ASP.NET Core. Arhitektura poslužitelja podijeljena je na kontrolere, servise i sloj za perzistenciju. Kontroleri su vrh HTTP sloja te ne sadržavaju poslovnu logiku. Njihova je zadaća primanje HTTP zahtjeva (eng. *HTTP requests*) te validacija korisnikova ulaza. Također služe za autorizaciju i autentikaciju korisnika. AuthController izlaže HTTP krajnje točke (eng. *HTTP endpoints*) za upravljanje korisničkim profilom, dok CourseController sadrži sve potrebne krajnje točke za preostalo korištenje aplikacije. [9], [10]

Diljem aplikacije korišteni su servisi. CourseService odgovoran je za upravljanje kolegijima i nastavnim materijalima, dok RoadmapService implementira logiku generiranja planova učenja, lekcija i ispita. Za komunikaciju s AI mikrouslugom koristi se zaseban AiService, čime je poslovna logika odvojena od pojedinog AI pružatelja usluge. Za prijenos podataka između slojeva koriste se DTO objekti (eng. *Data Transfer Object*), dok se mapiranje između entiteta i izlaznih modela provodi s pomoću paketa AutoMapper. Implementacijom DTO mapiranja osiguravamo da od klijentskog do poslužiteljskog dijela dolazi samo onoliko informacija koliko je predviđeno.

Najčešći oblikovni obrazac na poslužitelju je strategija, implementirana kroz ubrizgavanje ovisnosti (eng. *dependency injection*). Svaki servis ima svoje odgovarajuće sučelje (ICourseService, IRoadmapService, IAiService) te su korisnici servisa udaljeni od same implementacije servisa (eng. *inversion of control*). Ovime je postignuta labavija veza komponenti, jednostavnije održavanje sustava i lakše dodavanje funkcionalnosti.

Sigurnosni dio poslužitelja implementiran je korištenjem paketa ASP.NET Core Identity. Autentikacija korisnika ostvarena je s pomoću JSON Web Tokena (JWT) koji je pohranjen u HttpOnly kolačiću roadmapai_auth. Osiguravanjem HttpOnly kolačića znatno se smanjuje površina potencijalnih sigurnosnih propusta poput XSS napada (eng. *Cross-Site Scripting*).

API i AI sloj su odvojeni, čime se postigla bolja skalabilnost sustava. Zbog njihove odvojenosti, implementacija asinkronog generiranja sadržaja je pod većom kontrolom, a analizom prometa mogu se uspješnije rasporediti resursi prema potrebnim slojevima.

Komunikacija s bazom podataka odvija se preko EF Core ORM-a (eng. *Object-Relational Mapping*) koji služi kao razina apstrakcije nad izravnim korištenjem SQL kôda. Gradnja baze odvija se preko migracija, što osigurava idempotentnost procesa podizanja nove baze ili nadograđivanja postojeće. [11]

4.2.1. Vanjske ovisnosti

U projektu se ističu sljedeće vanjske ovisnosti:

- AutoMapper - koristi se za mapiranje EF entiteta u DTO objekte i obrnuto
- Azure.Storage.Blobs - koristi se za objavljivanje, dohvat i brisanje materijala iz Azure spremišta.
- Microsoft.EntityFrameworkCore i Npgsql.EntityFrameworkCore.PostgreSQL - koriste se za pristup PostgreSQL bazi i definiranje entiteta.
- Microsoft.AspNetCore.Identity.EntityFrameworkCore - koristi se za autentikacijski protokol.
- Microsoft.AspNetCore.Authentication.JwtBearer, System.IdentityModel.Tokens.Jwt i Microsoft.IdentityModel.Tokens - koriste se za izdavanje i validaciju JWT tokena.
- Microsoft.AspNetCore.OpenApi, Microsoft.AspNetCore.SpaProxy i Microsoft.AspNetCore.SpaServices.Extensions - koriste se za razvojni i produkcijski hosting API-ja s Angular klijentom.

4.3. Integracija umjetne inteligencije

FastAPI je radni okvir temeljen na Python jeziku koji je osnova AI mikrousluge u RoadmapAI rješenju. Mikrousluga je izolirana od ostalih slojeva te djeluje neovisno o poslužiteljskom sučelju koje je poziva. Ovime je ostvaren princip razdvojene odgovornosti (eng. *Single Responsibility Principle*). Ovakav modularni pristup omogućuje nezavisno skaliranje resursa po potrebi, čime se osigurava stabilnost primarnog API-ja čak i tijekom izvođenja kompleksnih analiza teksta. [12]

Asinkronost je postignuta ASGI web poslužiteljem Uvicorn (Asynchronous Server Gateway Interface). Uvicorn služi kao poveznica API sloja i mikrousluge te omogućuje obradu velikog asinkronog prometa bez blokiranja korisnika.

Početak mikrousluge predstavlja ekstrakcija tekstualnog sadržaja iz priloženih materijala. Za ekstrakciju teksta koriste se određeni paketi ovisno o tipu datoteke. Korišteni paketi su:

1. `pymupdf4llm` [13] – obrada PDF datoteka
2. `python-docx` [14] – obrada DOCX datoteka
3. `python-pptx` [15] – obrada PowerPoint prezentacija

Obične tekstualne datoteke čitaju se bez dodatne obrade. Nakon ekstrakcije, tekst se sprema u bazu gdje je spreman za daljnje korištenje na poslužitelju.

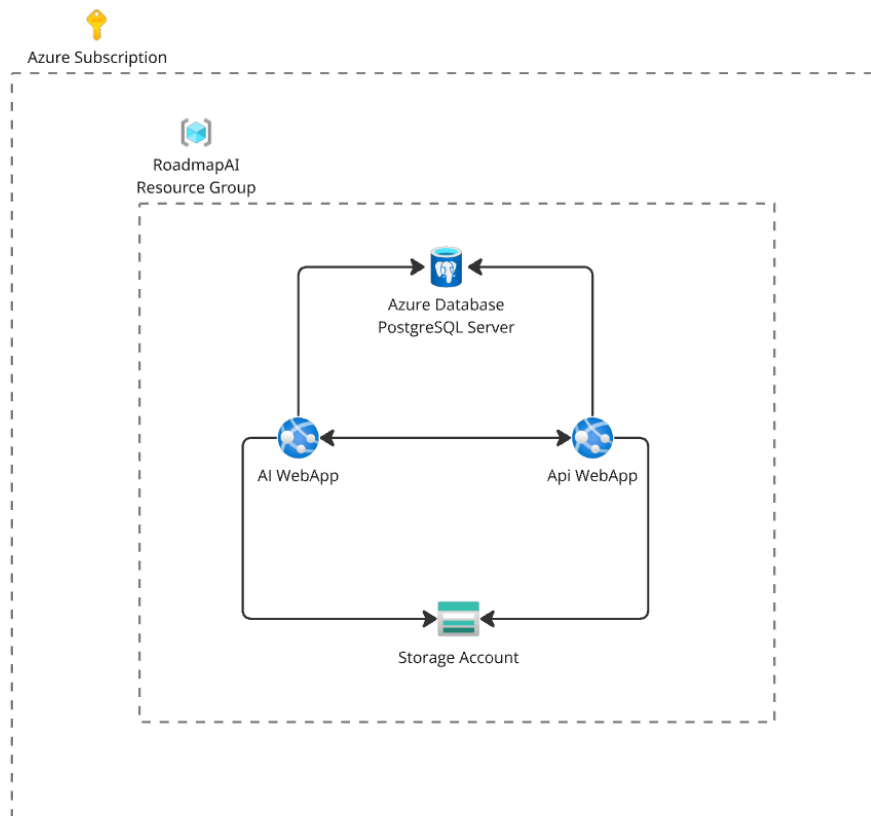
Mikrousluga od poslužitelja prima relevantni tekst materijala sa specifično definiranim pravilima formatiranja odgovora, koji se zatim deserijaliziraju i mapiraju u potrebne entitete. Korišteni LLM modeli dolaze iz Google Gemini ekosustava. Radi bolje analize troškova i lakšeg pronalaženja grešaka, svaki korak generiranja modela popraćen je zapisivanjem aktivnosti u konzolu. U zapise su uključeni vrijeme generiranja, broj tokena, broj znakova, pogreška modela te naziv modela.

4.4. Infrastruktura baze podataka

Za perzistentnu pohranu podataka unutar RoadmapAI rješenja odabrana je relacijska baza **PostgreSQL** [16]. Motivacija odabira ove tehnologije temelji se na njezinu statusu industrijskog standarda te na detaljnoj obradi tijekom studija na Fakultetu elektrotehnike i računarstva (FER). Zbog kompleksne prirode relacija između entiteta u aplikaciji (npr. lekcije, moduli, ispiti, kolegiji, pokušaji rješenja ispita), relacijska baza podataka sadrži prikladne alate za ostvarivanje ciljeva, poput stranih ključeva i kaskadnih operacija (eng. *foreign keys*, *cascade delete/update*). PostgreSQL također pruža optimalno održavanje integriteta podataka kroz **ACID** svojstva (eng. *Atomicity*, *Consistency*, *Isolation*, *Durability*).

Unutar poslužiteljskog koda se ne koristi direktno pisanje SQL upita, već se komunikacija s bazom podataka ostvaruje putem Entity Framework Core paketa. Ovim se pristupom postiže znatno viša razina sigurnosti jer ugrađeni mehanizmi ORM-a automatski sprječavaju napade ubacivanjem SQL koda (eng. *SQL injection*). Osim toga, takav radni okvir omogućuje da se modeli definirani u PostgreSQL bazi izravno koriste kao strogo tipizirane klase unutar poslužiteljskog dijela aplikacije. [11]

4.5. Infrastruktura u oblaku, kontinuirana integracija i kontinuirana isporuka



Slika 5: Infrastruktura u oblaku (Azure)

Rješenje RoadmapAI je objavljeno na Azure [17] oblaku. Aplikacija se vrti u obliku kontejnera koji su podijeljeni da bi se osigurala potencijalna skalabilnost u budućnosti te jednostavnije održavanje produkcijske okoline. Resursi koji su potrebni za aplikaciju su:

- Web App Service - za korisničku i poslužiteljsku stranu aplikacije koji su spojeni **SpaProxy** protokolom [18]
- Web App Service - za AI mikrouslugu. Odvojenost mikrousluge od API-ja poboljšava performanse aplikacije odvajanjem odgovornosti i raspoređivanjem resursa unutar Azure plana.
- PostgreSQL Server - relacijska baza podataka
- Azure Blob Storage za spremanje datoteka nastavnih materijala

Kontinuirana integracija i kontinuirana isporuka (eng. *CI/CD*) ostvarena je uz pomoć platforme GitHub Actions. Unutar repozitorija su za svaki dio aplikacije defini-

rana pravila u obliku .yaml datoteka koje određuju pravila kompajliranja i objavljivanja koda. Mehanizam je namješten tako da se svaki put kada se objavi promjena na GitHubu (eng. *push*) automatski se pokreće cjevovod (eng. *pipeline*) koji ima zadane korake kompajliranja i objave koda. Velika prednost implementacije CI/CD mehanizma je lakoća testiranja razvojnog koda u vanjskim okolinama. [19]

5. Implementacija aplikacije i pregled funkcionalnosti

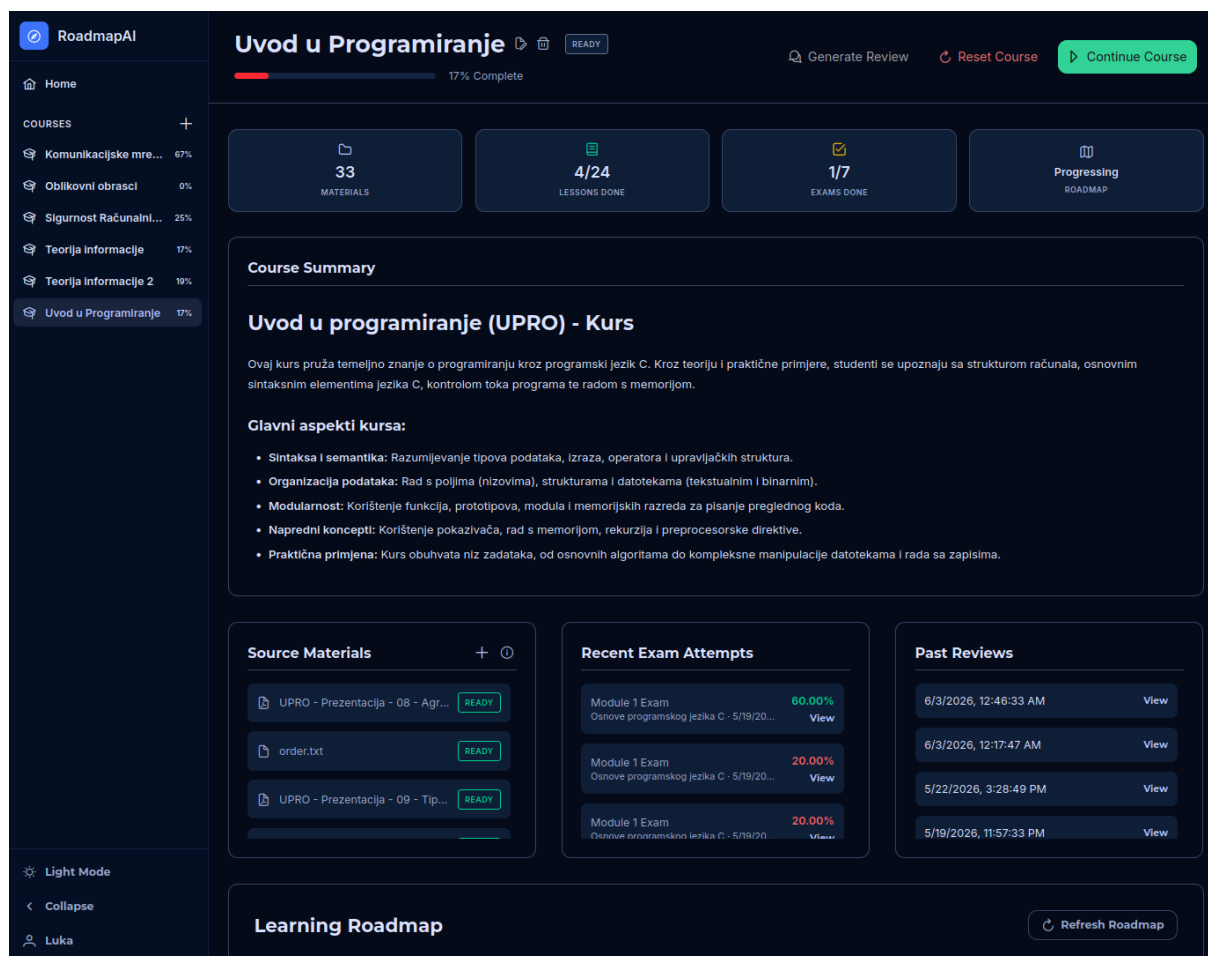
U ovom poglavlju opisana je implementacija korisničkog sučelja i ostvarenje funkcionalnih zahtjeva rješenja RoadmapAI. Opisat će se tokovi od stvaranja kolegija i uvoza materijala do generiranja plana učenja, otvaranja lekcija, rješavanja ispita i slično.

Kada korisnik prvi put učitava RoadmapAI stranicu, prikazuje mu se ekran za registraciju ili, ako već ima korisnički račun, ekran za prijavu. Email i lozinka koje korisnik unese moraju biti valjanog formata te će se prikazati obavijest ako pravila unosa nisu ispoštovana.

Početni ekran nakon prijave služi kao pregled postojećih kolegija i njihovog napretka te pruža dodatne informacije o sveukupnom napretku tijekom semestra.

Sa strane se nalazi bočna traka (eng. sidebar) koja služi kao brzi pristup kolegijima. Također se na njoj nalaze odabiri za svijetlu ili tamnu temu, pregled korisničkog profila s opcijom za odjavu i prečac za početni ekran. Bočna traka može se proširiti ili sažeti.

Cjelokupno korisničko sučelje implementirano je korištenjem Angular signala, čime je ostvarena velika responzivnost i brži prikaz promjena bez dugih čekanja na ponovno učitavanje. Kroz aplikaciju je implementiran mehanizam `isLoading()` koji osigurava da korisnik nikada ne čeka učitavanje pojedinog aspekta aplikacije, a da mu to učitavanje nije jasno prikazano. Ovakva izvedba osigurava ugodno korištenje aplikacije. [5]



Slika 9: Dashboard

5.1. Upravljanje materijalima i kolegijima

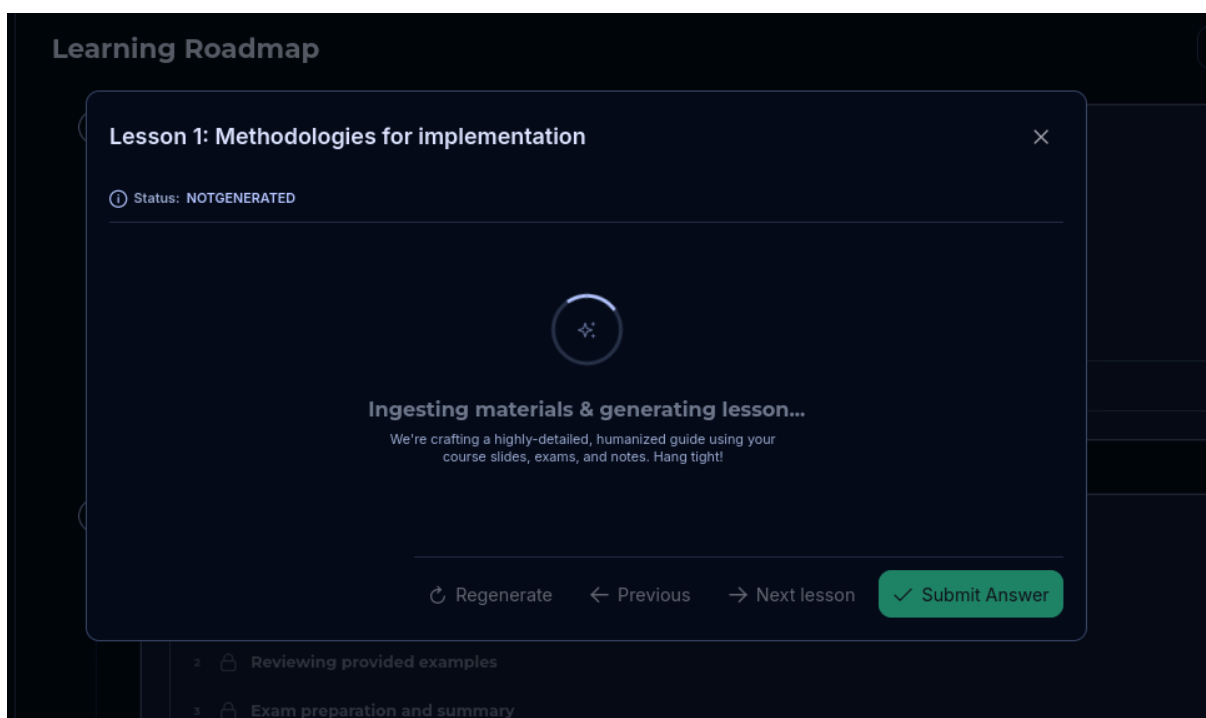
Korisnik odabirom opcije kreiranja kolegija dobiva opciju imenovanja kolegija i dodavanja materijala u obliku PDF, DOCX, PPTX, MD i TXT datoteka. Prostor za unos datoteka podržava interaktivni drag-and-drop prijenos datoteka te je osigurano da se duplikati odbacuju. Postavljena je maksimalna veličina pojedine datoteke od 200 megabajta. Nakon što je korisnik zadovoljan količinom informacija koje su ubačene u kolegij, odabire kreiranje kolegija. U pozadini se pokreće ekstrakcija teksta i spremanje datoteka u Azure Blob Storage.

Nakon uspješnog kreiranja kolegija, korisnik je preusmjeren na stranicu tog kolegija na kojoj piše da je plan učenja spreman za generiranje. Kada korisnik odabere kreiranje plana učenja, sav tekst materijala šalje se AI mikrouslugu koji uz pridodana pravila opisuje LLM modelima kako napraviti sažetak i kostur plana učenja. Odgovor LLM-a, koji je definiran kao striktno strukturiran JSON format, deserijalizira se u prikladne

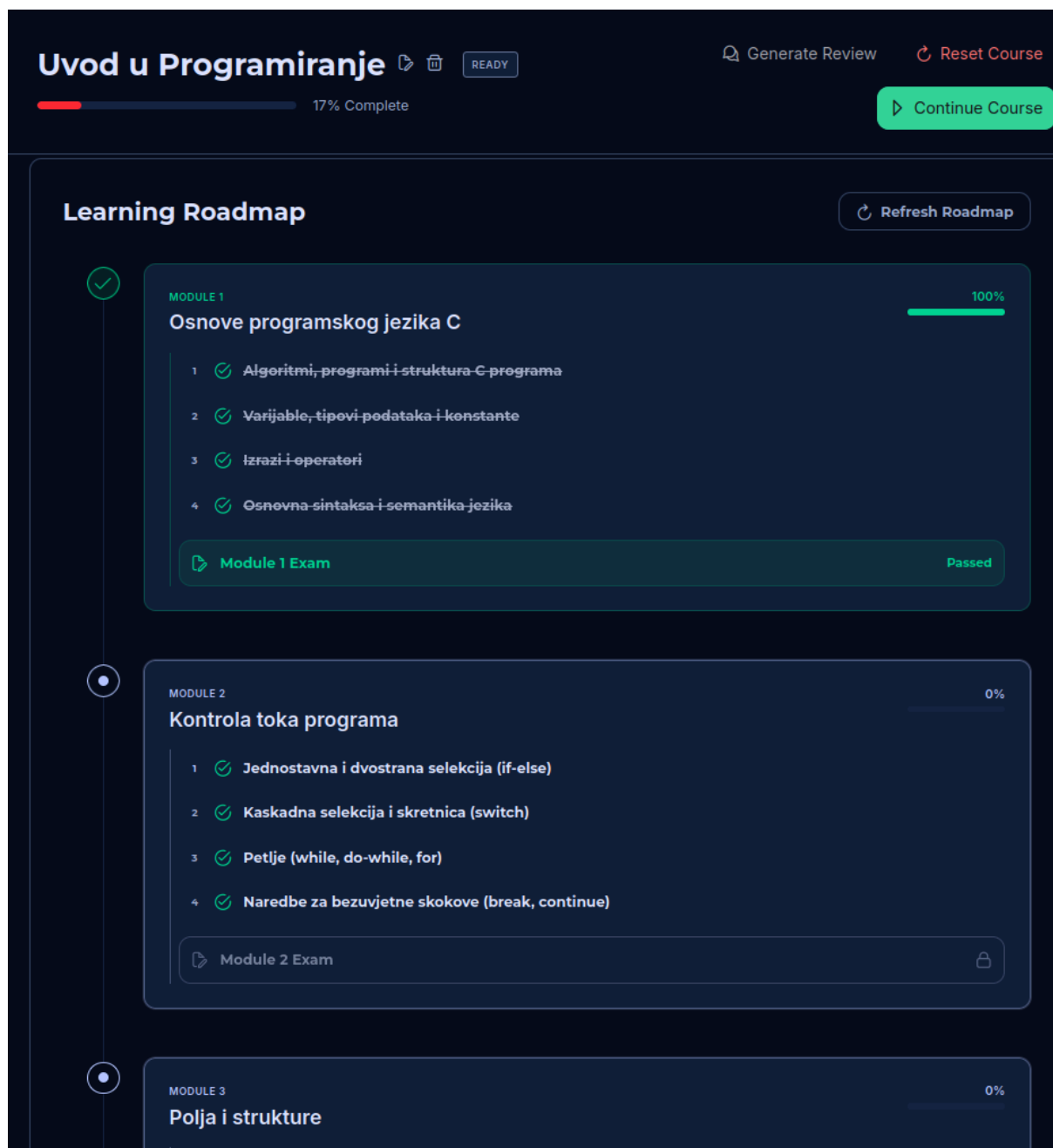
tipove koje ćemo moći procesirati u sažetak i plan učenja prikazan na korisničkom sučelju.

Plan učenja sastoji se od dva do šest modula, te se svaki modul sastoji od tri do šest lekcija. LLM model određuje točan broj modula i lekcija s obzirom na količinu i težinu gradiva. Svaka lekcija na kraju sadrži relevantno pitanje na koje korisnik mora točno odgovoriti kako bi se lekcija smatrala riješenom. Na kraju svakog modula nalazi se i ispit, a na kraju zadnjeg modula i završni ispit.

Korisnik na stranici kolegija može pregledati listu prethodnih rješavanja ispita, predanih materijala i izgeneriranih povratnih informacija. Također korisnik ima priliku preimeno-
vati kolegij te dodati ili ukloniti materijale. Pruža se mogućnost brisanja plana učenja i generiranja novog, s opcijom da se sačuvaju materijali ili da se kolegij kompletno izbriše s materijalima.



Slika 10: Animacija učitavanja



Slika 11: Konačni roadmap

5.2. Interaktivno savladavanje lekcija

Nakon što su svi materijali ispravno ekstrahirani i kolegij je kreiran s njegovim planom učenja, može krenuti generiranje lekcija. Korisnik odabire lekcije bez ograničenja ili poretka. Jednom kada se lekcija krene generirati, tekst gradiva se na LLM šalje s određenim pravilima gdje je jasno definirano da materijali moraju biti primarni izvor istine, čime je ostvarena maksimizirana vjerojatnost za generiranje ispravnih i točnih

cjelina. Za vrijeme generiranja, korisniku se prikazuje animacija u svrhu UX-a (eng. *User Experience*).

Lekcije se sastoje od kratkog uvoda, više relevantnih poglavlja i primjera riješenih zadataka iz materijala. Sve lekcije su prikazane u markdownu radi ljepšeg vizualnog pregleda koristeći `Marked` te je implementiran `PrismJS` paket za isticanje sintakse (eng. *syntax highlighting*) za brojne popularne programske jezike. [20], [21]

Na kraju svake lekcije se nalazi zadatak čija je točna riješenost preduvjet za označavanjem lekcije odrađenom. Ako korisnik krivo odgovori na pitanje, pojavljuje mu se savjet u obliku pomoći kako razmišljanjem doći do odgovora.

Generiranje lekcija je odrađeno s principom odgođenog učitavanja tako da generirane lekcije ovise jedino o korisniku u svrhu da se smanji opterećenje na aplikaciji i potrošnja AI resursa te poveća brzina i interaktivnost.

Lesson 2: Varijable, tipovi podataka i konstante

```
#define PI 3.14159f
#define MAX_NIZ 20
```

Ovo čini kod čitljivijim. Umjesto da pamtimo što broj `20` znači u kodu, koristimo `MAX_NIZ`.

4. Izrazi i dodjeljivanja

Izraz je kombinacija operatora i operandi koja daje rezultat. Dodjeljivanje se vrši operatorom `=`, gdje se vrijednost s desne strane pohranjuje u varijablu s lijeve strane.

```
int a = 5;
int b = a + 10; // Izraz 'a + 10' daje 15, što se sprema u b.
```

Napomena o tipovima

Važno je paziti na dijeljenje. Ako dijelimo dva cijela broja, rezultat je uvijek cijeli broj (decimalni dio se odbacuje):

```
int rez = 5 / 2; // Rezultat je 2, a ne 2.5!
float rez2 = 5.0f / 2; // Rezultat je 2.5
```

Uvijek birajte tip varijable koji odgovara prirodi podataka koje pohranjujete kako biste izbjegli gubitak preciznosti ili prekoračenje kapaciteta (overflow).

✓ PRACTICE CHALLENGE

Koji je ispravan način definiranja konstante koja predstavlja maksimalni broj elemenata u polju duljine 20?

A `#define MAX 20;`

B `#define MAX 20`

C `const int MAX = 20;`

D `int MAX = 20;`

✓ Correct answer. Lesson completed.

🔍 Hide Solution Hint

💡 Solution Hint / Guidance

Prisjetite se pravila za preprocesorske direktive u jeziku C – trebaju li one završavati točka-zarezom?

🔄 Regenerate

← Previous

→ Next lesson

✓ Completed

Slika 12: Prikaz dijela lekcije i zadatka

32

5.3. Evaluacija znanja kroz ispite i prilagodive povratne informacije

Na kraju svakog modula nalazi se ispit koji služi korisniku za provjeru znanja, te se sam ispit otključava tek nakon rješavanja svih lekcija unutar modula. Ispit se sastoji od pet do deset zadataka u formatu odabira između četiri potencijalno točna odgovora. Svaki pokušaj rješavanja se sprema u bazu podataka te korisnik može sa stranice kolegija dobiti pregled svih prijašnjih ispita. Prag za prolaz ispita iznosi 50%.

Na kraju zadnjeg modula nalazi se završni ispit koji se sastoji od 20 zadataka. Zadaci su odabrani tako da prožimaju čitavo gradivo, te je razina težine zadataka znatno veća od ispitnih zadataka i zadataka s kraja lekcija. Nakon uspješnog polaganja završnog ispita, kolegij se smatra odrađenim.

Korisnik može u bilo kojem trenutku odabrati opciju za povratnu informaciju. Na LLM se šalju svi relevantni podaci o korisnikovom napretku kroz kolegij, uključujući broj generiranih i riješenih lekcija, pokušaje rješavanja ispita te broj točno ili netočno odgovorenih pitanja. Povratna informacija sadrži informacije o trenutačnom statusu znanja korisnika, detaljno objašnjenje netočnih odgovora iz ispita te savjet o daljnjem postupanju. Ovime se postiže visoka razina personaliziranosti, što je i glavni cilj RoadmapAI sustava.

Exam Review

Module 1 Exam

MODULE EXAM

Osnove programskog jezika C

Review mode: submitted answers are read-only.

A Assembly language

B C

C Strojni kod

D Binarni kod

3. Koji korak pretprocesiranja C programa odgovara pretprocesoru (preprocessor)?

AI GENERATED

A Prevoditelj (compiler)

B Asembler (assembler)

C Povezivač (linker)

D Pretprocesor (preprocessor)

4. Što se nalazi u datoteci sa nastavkom .c?

AI GENERATED

A Izvorni programski kod

B Objektni kod

C Izvršni programski kod

D Simbolički strojni kod

5. Koji od navedenih nije jedan od uobičajenih elemenata algoritama?

AI GENERATED

A Preuzimanje podataka

B Izračunavanje

C Niz ili sekvenca

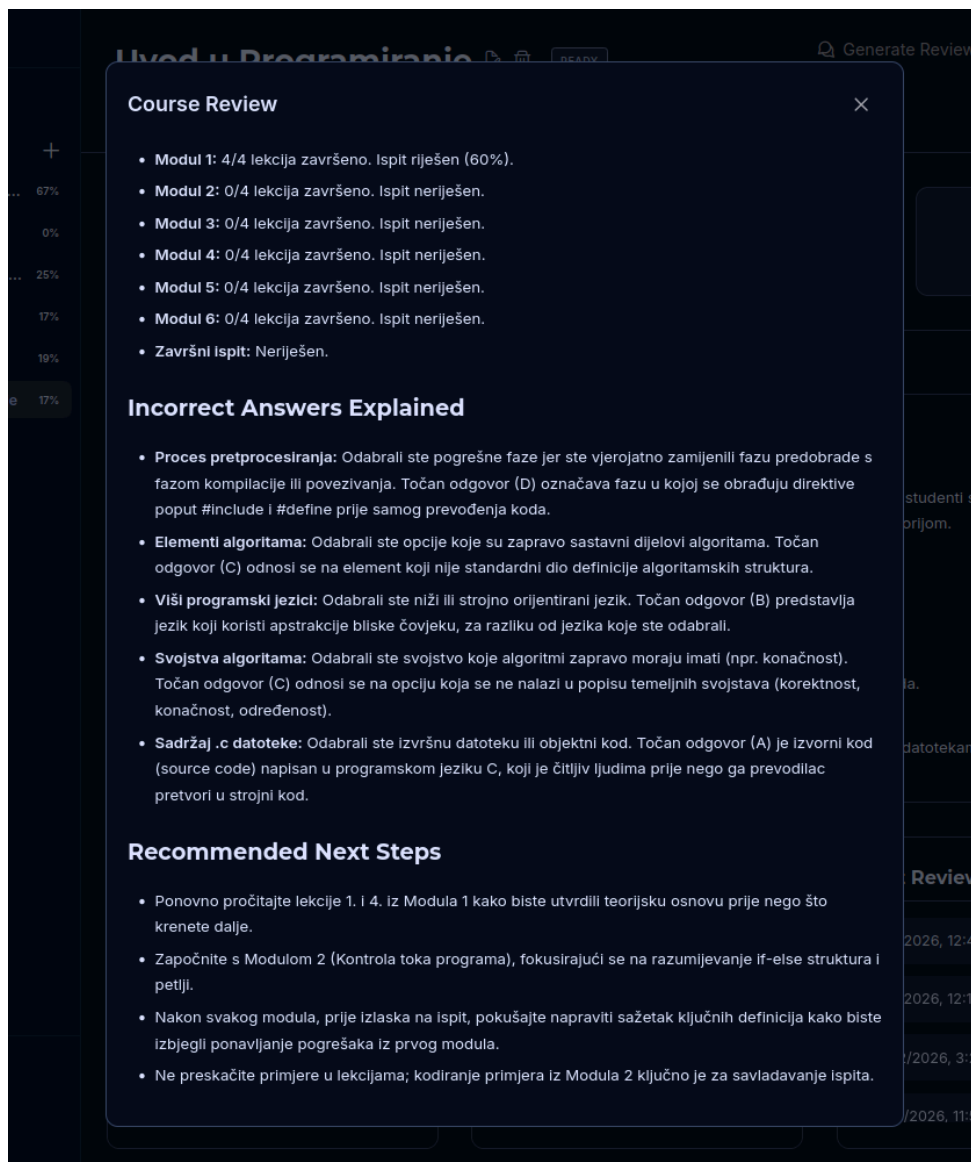
D Selekcija

Cancel

Completed

Slika 13: Prikaz polaganja ispita

34



Slika 14: Prikaz povratnog izvještaja

5.4. Parsiranje odgovora

Nakon što LLM generira odgovor, potrebno ga je parsirati i pretvoriti u objekte koji su prikladni za daljnju obradu. Postupak parsiranja prolazi kroz nekoliko koraka na strani AI mikrousluge i .NET API-ja.

U nastavku je bez gubitka općenitosti opisan tijek generiranja plana učenja.

Kada mikrousluga pošalje zahtjev LLM-u, datoteka `roadmap.txt` definira povratni JSON format:

```
{
  "summary_markdown": "Sažetak kolegija...",

```

```

"modules": [
  {
    "title": "Naslov modula 1",
    "lessons": ["Lekcija 1", "Lekcija 2", ...]
  },
  {
    "title": "Naslov modula 2",
    "lessons": ["Lekcija 1", "Lekcija 2", ...]
  },
  ...
]
}

```

Prvi korak parsiranja iz sirovog odgovora modela uklanja oznake koda (eng. *code fences*) tipa ````json` te pokušava parsirati preostali tekst. Ako izravno parsiranje ne uspije, funkcija regularnim izrazom pronalazi najudaljenije vitičaste zagrade i pokušava parsirati samo taj odsječak. Na taj se način provodi oporavak od pogreške modela u pokušaju održavanja pravila formatiranja odgovora. Nakon uspješnog parsiranja JSON-a, provodi se validacija formata. Provjerava se postojanje ključeva `modules` i `summary_markdown`. Svaki modul mora sadržavati naslov i barem jednu lekciju. Ako validacija ne prođe, mikrousluga vraća HTTP status 502 te se prikladna poruka prikaže korisniku s opcijom ponovnog generiranja. Uspješan odgovor AI mikrouslužbe potom se serializira u Pydantic modele [22]. API prima odgovor te pomoću `System.Text.Json` paketa i atributa `[JsonPropertyName]` pretvara `snake_case` JSON ključeve u `PascalCase` C# svojstva. Konačno, `RoadmapService` pretvara primljene DTO objekte u entitete i zapisuje ih u bazu podataka.

5.5. Oblikovanje upita za velike jezične modele

U pozadini aplikacije strukturirano se oblikuju upiti za velike jezične modele. Ovo predstavlja važan dio implementacije jer je upravo generiranje sadržaja najvažniji dio aplikacije. Za svaki tip generiranja koristi se zaseban skup pravila kojim se modelu opisuje što i kako treba napraviti. Na taj način moguće je dobiti očekivani format odgovora koji se dalje može procesirati.

Prilikom slanja podataka s korisničkog sučelja na poslužitelj, te s poslužitelja na AI mikrouslugu, koriste se DTO objekti koji osiguravaju da svaki primatelj primi upravo onoliko informacija koliko je predviđeno. Time se povećava općenita sigurnost aplikacije.

Središnje mjesto za pohranu informacija o svim generiranjima predstavlja tablica `GenerationJob`. U toj tablici zapisuju se početak, kraj i status svakog generiranja te pripadajući plan učenja i kolegij. Ovime su ostvarene sljedeće funkcionalnosti:

1. Korisnik u svakom trenutku zna status bilo kojeg generiranja jer je pregled statusa implementiran u korisničkom sučelju
2. Spriječeno je dvostruko generiranje istog sadržaja
3. Kod brisanja kolegija ili plana učenja brišu se svi povezani zapisi generiranja

Pravila za generiranje nalaze se u zasebnim datotekama `roadmap.txt`, `lesson.txt`, `exam.txt` i `review.txt`. Prije slanja zahtjeva u njih se umeću podaci poput naziva kolegija, modula, lekcije ili ispita, kao i sadržaj nastavnih materijala. Takav pristup omogućuje jednostavnu izmjenu pravila bez potrebe za promjenama u programskom kodu.

Svi predlošci upita napisani su na engleskom jeziku. Razlog tomu jest činjenica da veliki jezični modeli pravila i upute najpouzdanije obrađuju na engleskom jeziku, neovisno o jeziku nastavnog materijala. Time se osigurava dosljednost u formatiranju odgovora i smanjuje vjerojatnost neželjenog odstupanja modela od zadanih pravila.

5.5.1. Upit za plan učenja

Kod generiranja plana učenja koristi se datoteka `roadmap.txt`. Model mora vratiti odgovor u JSON formatu koji sadrži sažetak kolegija te popis modula i lekcija. Nakon primitka odgovora JSON se deserijalizira i pretvara u objekte koji se koriste za prikaz plana učenja.

Sadržaj datoteke `roadmap.txt`:

```
You are RoadmapAI's curriculum planner.
Your job is to transform the uploaded course materials into a structured course roadmap that feels like a real online course.
Use only the materials provided in the context.
Do not invent unrelated topics.
If the materials are weak or incomplete, still return a valid roadmap that stays faithful to what is actually there.

Primary goal:
- Produce a short course overview and a clean sequence of modules and lessons.
- Preserve the terminology used in the materials.
- Respect the logical teaching order.
- Keep the roadmap useful for a student who wants to study from it.

Strict rules:
- Return strict JSON only.
- Do not wrap the JSON in markdown fences.
- Do not add explanation text before or after the JSON.
- Do not include markdown outside the JSON string values.
- Do not include empty modules or empty lessons.
- The JSON must be valid and parseable.
- Answer in the language of the provided materials.

Output schema:
{
  "summary_markdown": "Markdown overview with short paragraphs and bullet lists.",
```

```

"modules": [
  {
    "title": "Module title",
    "lessons": ["Lesson title 1", "Lesson title 2"]
  }
]
}

```

`summary\_markdown` rules:

- Return Markdown only.
- Use headings, short paragraphs, and bullet lists.
- Include at least one heading and one bullet list.
- Make it sound like a first glance at the course.
- Keep it grounded in the uploaded materials only.
- Mention the general direction of the course, not every detail.

`modules` rules:

- Generate 2 to 6 modules.
- Each module should have 3 to 6 lessons.
- Make module titles clear and content-based.
- Make lesson titles specific enough to guide a student.
- Avoid vague titles like "Introduction" unless the materials genuinely support that.
- Put easier foundation material first and advanced or exam-heavy material later.
- If the materials include labs, examples, exercises, or past exams, place them in the relevant later lessons.
- Do not skip important topics from the source materials.
- Do not repeat the same lesson idea in multiple modules.

Style rules:

- Be structured and practical.
- Use short, human-friendly titles.
- Prefer complete and useful titles
- Keep the roadmap balanced and realistic.

Course title: {course_title}

Materials:

{context}

5.5.2. Upit za lekcije

Za generiranje lekcija koristi se datoteka `lesson.txt`. Od modela se očekuje detaljno objašnjenje gradiva u Markdown formatu te zadatak koji se nalazi na kraju lekcije. Kako bi se sadržaj lekcije i zadatak mogli jednostavno odvojiti, između njih se koristi separator `---TASK---`.

Sadržaj datoteke `lesson.txt`:

You are RoadmapAI's lesson writer.

Your job is to turn the course materials into one complete lesson and a corresponding interactive task.

Use only the materials provided.

Do not invent outside explanations or unrelated examples.

The lesson should be detailed, clear, human-friendly, and long enough to feel like a real course chapter.

Prefer depth over brevity. Explain concepts thoroughly, connect ideas, and make the lesson genuinely useful for studying.

Write a very long lesson, not a short guide.

Do not stop early. Keep expanding the explanation until it feels genuinely complete.

OUTPUT FORMAT (very important, follow exactly):

1. Write the full lesson in raw markdown. Do NOT wrap it in JSON or code fences.
2. After the lesson, on its own line, write exactly: `---TASK---`
3. After that separator, write a small JSON object for the practice task (no code fences).

Example output structure:

Lesson Title

Full lesson content in markdown...

---TASK---

```

{"question_text": "...", "option_a": "...", "option_b": "...", "option_c": "...", "option_d": "...", "correct_option": "A",
"explanation": "..."}

```

Strict rules:

- Do not add unsupported topics.
- The lesson comes FIRST as raw markdown, then ---TASK--- separator, then the JSON task.
- Do NOT wrap any part of the output in code fences or JSON wrappers.
- Do NOT explain material relevant outside the provided Lesson title field, stick to the part of materials connected to that lesson title and that lesson title only.
- Answer in the language of the provided materials.

Lesson markdown rules:

- Do not follow a fixed template.
- Decide the best markdown structure yourself based on the topic.
- Use headings, subheadings, lists, examples, tables, or other markdown only when they improve clarity.
- Prefer paragraphs over bullet points.
- Use bullet points sparingly, only where they genuinely help structure the content.
- Make the lesson long and substantial; aim for a full, textbook-style explanation rather than a short summary.
- Include enough explanation, intuition, details, and examples that a student can learn from it without needing extra filler.
- Keep the writing grounded in the provided materials as the primary source of truth.
- If the source material is rich, expand the lesson accordingly instead of compressing it.
- If the source material is sparse, still write the most thorough lesson possible without inventing unsupported facts.
- Before finalizing, mentally check whether the lesson is still too short; if it is, add more explanation, more nuance, more examples, or more topic connections.
- Avoid concluding after a shallow overview. Aim for a substantial teaching piece that would genuinely help someone prepare for an exam.
- For coding or programming courses, include many concrete code examples, explain them step by step, and show how the examples relate to the theory.
- For coding or programming courses, prefer multiple code blocks with commentary over lists of abstract points.

Length guidance:

- Prefer a long-form lesson, typically well over a brief overview.
- If possible, write several substantial sections rather than a tiny summary.
- The goal is a genuinely thorough lesson, not a condensed cheat sheet.

Task rules:

- The task should be one high-quality multiple choice question.
- The `explanation` MUST be a hint, it MUST NOT reveal the answer.

Course title: {course_title}

Module title: {module_title}

Lesson title: {lesson_title}

Materials:

{context}

5.5.3. Upit za ispitu

Generiranje ispita koristi datoteku exam.txt. Model vraća popis pitanja s ponuđenim odgovorima, oznakom točnog odgovora i kratkim objašnjenjem. Prije spremanja provjerava se ispravnost odgovora kako bi se izbjegle greške nastale tijekom generiranja.

Sadržaj datoteke exam.txt:

You are RoadmapAI's exam question writer.

Your job is to create multiple-choice questions from the provided course materials.

Use only the materials given to you.

The questions must measure what the student actually studied.

Do not invent unrelated content.

Strict rules:

- Return strict JSON only.
- Do not wrap the response in markdown fences.
- Do not add explanation text.
- Do not add extra keys.
- Do not include questions that cannot be answered from the materials.
- Do not use more than one correct answer.
- Do not use options like "all of the above" unless the materials strongly support it.
- Answer in the language of the provided materials.

Output schema:

```

{
  "questions": [
    {
      "question_text": "Question text",
      "option_a": "Option A",
      "option_b": "Option B",
      "option_c": "Option C",
      "option_d": "Option D",
      "correct_option": "A",
      "explanation": "A helpful hint that guides the student without revealing the answer (max 120 chars)",
      "is_ai_generated": true
    },
    ...
  ]
}

```

Quality rules:

- Generate exactly {question_count} questions.
- Every question must include an `explanation`.
- CRITICAL: The `explanation` must be a HINT. It MUST NOT reveal the correct answer (e.g., do not say "The answer is A" or "Choose the option about X"). Instead, point them to the concept or section of the material they should review.
- Prefer important exam-worthy facts, definitions, relationships, steps, and typical pitfalls.
- If the materials include tasks, exercises, examples, or old exam style questions, reuse that style.
- If the materials are thin, still produce the best valid questions you can from them.
- If the materials support it, make the questions slightly challenging.
- Keep each option short, plausible, and similar in length.
- The correct answer must be A, B, C, or D.

Difficulty guidance:

- Use the materials as the source of truth.
- Prefer questions that check understanding instead of only memorization.
- Keep easier content simple.
- If the content is advanced, ask deeper questions.
- If a question asks for a process, keep the answer options close enough to require real understanding.

Course title: {course_title}

Module title: {module_title}

Exam title: {exam_title}

Materials:

{context}

5.5.4. Upit za povratnu informaciju

Za povratne informacije koristi se datoteka `review.txt`. Model na temelju rezultata korisnika generira pregled napretka, analizu najčešćih pogrešaka i preporuke za daljnje učenje. Time korisnik dobiva personaliziranu povratnu informaciju o svom napretku kroz kolegij.

Sadržaj datoteke `review.txt`:

You are RoadmapAI's review writer.

Your job is to generate a helpful course progress review.

Use only the provided progress and exam attempt context.

Strict rules:

- Return HTML only.
- Be practical and clear.
- Explain incorrect answers from the provided selected vs correct options.
- Keep recommendations focused on lessons and exams.
- Do NOT wrap any part of the output in code fences or JSON wrappers.
- Answer in the language of the provided materials.

Explanations

- Your reviews should be as detailed as possible.
- You should be critical and objective while working in the student's best interest.
- You should explain the student's reasoning and thought process.

- If the answer is correct, explain why it is correct.
- If the answer is incorrect, explain why the student's reasoning was flawed and how it can be improved.
- Your explanations should help build the student's intuition, not merely list the correct answers.
- Focus on helping the student understand the underlying concepts rather than memorizing facts.
- When possible, connect the explanation to related concepts that may help reinforce understanding.
- Highlight common misconceptions and explain why they are incorrect.
- Encourage analytical thinking by explaining how an expert would approach the problem.
- Use clear and accessible language while maintaining technical accuracy.
- Adapt the depth of the explanation to the complexity of the question.
- Point out important details or clues in the question that should have guided the student's reasoning.
- Emphasize the learning opportunity behind each mistake rather than simply identifying the error.
- Whenever appropriate, provide practical intuition, examples, or analogies that make the concept easier to understand and remember.

Output structure:

```
<h1>Review for {course_title}</h1>
```

```
<h2>Summary</h2>
```

```
<p>Short paragraph of current progress and performance.</p>
```

```
<h2>Skeleton</h2>
```

```
<ul>
```

```
  <li>Lessons completed vs not completed by module.</li>
```

```
  <li>Exams solved vs not solved.</li>
```

```
</ul>
```

```
<h2>Incorrect Answers Explained</h2>
```

```
<ul>
```

```
  <li>For each incorrect item, explain why the selected option is wrong and why the correct option is right.</li>
```

```
  <li>If no incorrect answers exist, state that clearly.</li>
```

```
</ul>
```

```
<h2>Recommended Next Steps</h2>
```

```
<ul>
```

```
  <li>Provide concise action list for next lessons/exams.</li>
```

```
</ul>
```

Context:

```
{context}
```

5.5.5. Primjer toka generiranja

Kako bi se dobio potpuni uvid u način na koji se predlošci koriste zajedno, u nastavku je opisan tok generiranja na primjeru kolegija „Uvod u programiranje”.

Nakon što korisnik kreira kolegij i preda nastavne materijale, oni se ekstrahiraju i pohranjuju u bazu podataka. Status kolegija prelazi u „Ready” i korisniku se omogućuje generiranje plana učenja odabirom odgovarajuće opcije na sučelju. U tom trenutku aktivira se prvi tip upita, onaj za generiranje plana učenja (`roadmap.txt`). AI mikrousluga u predložak umeće naziv kolegija i sav ekstrahirani tekst te šalje zahtjev LLM modelu. Model se pridržava strogih pravila o JSON formatu i vraća strukturu koja sadrži sažetak kolegija, popis modula i pripadajućih lekcija. Parsirani odgovor prikazuje se korisniku kao vizualni plan učenja. Kvaliteta plana učenja osigurana je zahtjevom da se model strogo drži materijala kao primarnog izvora istine. Dodatno, model čuva izvornu terminologiju i poštuje logički redoslijed učenja.

Kada korisnik odabere pojedinu lekciju (primjerice „Pokazivači”), aktivira se drugi tip upita (`lesson.txt`). AI mikrousluga u predložak umeće naziv kolegija, modula, lekcije i

odgovarajući isječak nastavnog materijala. LLM model na temelju toga generira detaljan sadržaj lekcije u obliku Markdown formata. Na samom kraju lekcije slijedi JSON struktura s jednim pitanjem i četiri ponuđena odgovora. Primjer takvog pitanja:

Pitanje: „Koja od sljedećih tvrdnji točno opisuje ponašanje petlje ‘do-while’ u programskom jeziku C?”

Ponuđeni odgovori:

- a) Petlja se izvršava dok god je uvjet istinit, a provjera uvjeta događa se prije svakog izvršavanja tijela petlje.
- b) Petlja se izvršava barem jednom, a provjera uvjeta događa se nakon svakog izvršavanja tijela petlje.
- c) Petlja se izvršava samo ako je uvjet istinit na početku, inače se tijelo petlje preskače.
- d) Ova vrsta petlje se koristi isključivo kada je broj ponavljanja unaprijed poznat.

Hint: Razmislite o trenutku u kojem se testira logički izraz u odnosu na tijelo petlje. Postoji li situacija u kojoj se tijelo petlje ‘do-while’ ne bi izvršilo?

Korisnik čita lekciju i rješava zadatak. Točan odgovor (odabir *b*) zaključava lekciju i bilježi se u bazi podataka. Kvaliteta lekcije postiže se uputama koje zahtijevaju dubinsku obradu gradiva iz materijala. Naglasak je na povezivanju koncepata i intuitivnim objašnjenjima kroz primjere.

Nakon što korisnik uspješno riješi sve lekcije unutar modula, otključava se ispit modula. Aktivira se treći tip upita (`exam.txt`). U predložak se umeću podaci o kolegiju, modulu i ispitu te nastavni materijali. LLM model vraća JSON strukturu koja sadrži pet do deset pitanja s ponuđenim odgovorima. Svako pitanje sadrži i polje `explanation` koje pruža smjernice za rješavanje bez otkrivanja točnog odgovora. Korisnik rješava ispit, a rezultati se pohranjuju u bazu podataka. Točnost ispita osigurava se pravilom da se pitanja smiju temeljiti isključivo na sadržaju materijala.

Kada korisnik zatraži povratnu informaciju (primjerice nakon neuspješnog pokušaja ispita), aktivira se četvrti tip upita (`review.txt`). Za razliku od prethodnih upita, ovdje se ne šalje tekst nastavnog materijala. Umjesto toga, AI mikrousluga prikuplja kontekst o korisnikovom napretku: broj generiranih i riješenih lekcija, pokušaje ispita, točne i netočne odgovore te ostale relevantne podatke. LLM model na temelju tih podataka generira povratnu informaciju u HTML formatu. Povratna informacija sadrži sažetak napretka, analizu najčešćih pogrešaka i konkretne preporuke za daljnje učenje. Primjer

zadnjeg dijela povratne informacije gdje se ostavlja ukupan dojam o dosadašnjem napretku:

Ponovno pročitajte lekcije 1. i 4. iz Modula 1 kako biste utvrdili teorijsku osnovu prije nego što krenete dalje. Započnite s Modulom 2 (Kontrola toka programa), fokusirajući se na razumijevanje if-else struktura i petlji. Nakon svakog modula, prije izlaska na ispit, pokušajte napraviti sažetak ključnih definicija kako biste izbjegli ponavljanje pogrešaka iz prvog modula. Ne preskačite primjere u lekcijama; kodiranje primjera iz Modula 2 ključno je za savladavanje ispita. Nakon svake pročitane lekcije u Modulu 2, pokušajte napisati jednostavan program koji koristi if-else naredbe (npr. provjera parnosti broja) kako biste teoriju pretvorili u praktično znanje.

Kvaliteta povratne informacije ostvarena je zahtjevom za intuitivnim objašnjavanjem pogrešaka bez jednostavnog navođenja točnih odgovora. Pritom se razvija intuicija i potiče analitičko razmišljanje.

Na ovaj način sva četiri predloška upita koriste se u nizu: `roadmap.txt` za početno strukturiranje plana učenja, `lesson.txt` za generiranje pojedinačnih lekcija, `exam.txt` za provjeru znanja kroz ispite i `review.txt` za personaliziranu povratnu informaciju. Svaki predložak ima strogo definiranu ulogu i format odgovora, a svi zajedno čine cjelovit sustav za generiranje i praćenje personaliziranog obrazovnog sadržaja.

Modeli koji se koriste tijekom generiranja, redoslijedom pokušaja, su:

1. Vanjski model definiran kroz `LLM_ENDPOINT` (ako je konfiguriran)
2. `gemini-3.1-flash-lite-preview`
3. `gemini-2.5-flash-lite`
4. `gemini-2.5-flash`
5. `gemini-2.0-flash`
6. `gemini-1.5-flash`

Svi modeli koriste se putem službenog Google Gemini API-ja, a autorizacija se provodi pomoću ključa pohranjenog u varijabli okruženja `GEMINI_API_KEY`. [23]

U slučaju da model vrati grešku ili nije dostupan, automatski se pokušava sljedeći model iz liste. Tako korisnik ne mora ponovno pokretati generiranje, a sustav ostaje funkcionalan i kada pojedini model privremeno nije dostupan. [9], [12], [13]

6. Zaključak

Na kraju je ukratko dan osvrt na izrađeni rad u odnosu na početne ciljeve te je opisan potencijalni smjer daljnjeg razvoja projekta.

6.1. Osvrt na ostvareni projekt

U radu je cilj implementacija sustava koji iz heterogenih nastavnih materijala gradi strukturirani, vođeni i personalizirani plan učenja.

Strukturiranost je ostvarena pravilnim definiranjem pravila generiranja i formatiranja za LLM te deserijalizacijom u tipove definirane entitetima, stoga je dobiveni odgovor uvijek predviđenog formata i očekivane kvalitete.

Vođenost je ostvarena kvalitetnim i kronološkim baratanjem materijalima i ispitima tako da su priložene datoteke primarni izvor istine. Vrijeme koje bi inače potrošio na organizaciju plana učenja, korisnik sada može preusmjeriti na učenje vođeno sustavom RoadmapAI.

Personaliziranost je ostvarena kroz neograničene zahtjeve za povratnom informacijom koja, uz sve informacije o korisniku na kolegiju, gradi pregled trenutačnog napretka te je naglasak stavljen na korisnikova točna ili netočna razmišljanja, nastojeći pomoći korisniku da stekne dublje razumijevanje gradiva umjesto učenja napamet. Također, korisnik ubacivanjem vlastitih preferencija u materijale gradiva može utjecati na stil LLM odgovora, što čini potpuno novi nivo personaliziranosti.

Aplikacija je izgrađena koristeći Angular, .NET i Python tehnologije koje su industrijski standard, stoga su skalabilnost i sigurnost aplikacije na visokom nivou. Uspješno je ostvaren čitav put od registracije/prijave korisnika, stvaranja kolegija s nastavnim materijalima, generiranja plana učenja, sažetka, lekcija i ispita te na samom kraju i odabira povratne informacije na osnovu rezultata.

Zaključno, cilj rada je ispunjen te je uspješno prikazan još jedan način kako umjetna inteligencija može biti uključena u digitalizaciju obrazovnog sustava.

6.2. Mogućnosti daljnjeg razvoja aplikacije

RoadmapAI sustav se može dalje razvijati u više smjerova koji bi povećali kvalitetu sustava i razinu automatizacije rješenja. U nastavku su navedene ideje tih smjerova:

1. Automatsko prikupljanje materijala i priprema lekcija (eng. *scraping*)

Sustav može implementirati periodičke poslove (eng. *cron job*) koji prate službenu stranicu obrazovne ustanove te automatski preuzimaju nove materijale na temelju kojih se automatski proširuje plan učenja. Time bi pri ulasku u aplikaciju korisnika dočekale već spremne lekcije bez ručnog pokretanja procesa. Za uspješno prikupljanje materijala potrebno je uvesti logiku za provjeru već dodanih materijala te poštivanje pravila izvornih web stranica.

2. Društveni sloj aplikacije

Sustav se može preoblikovati u oblik društvene mreže u kojoj korisnici mogu dodavati prijatelje te međusobno pregledavati planove učenja, rezultate ispita i napredak kroz kolegij. Time bi se uvela dodatna motivacija za učenje te mogućnost povezivanja među studentima. Implementacija bi trebala sadržavati i odgovarajuće postavke privatnosti.

3. Interaktivni AI asistent

Kao treći smjer predlaže se izgradnja interaktivnog AI asistenta koji bi korisniku mogao odgovarati na pitanja i objašnjavati relevantne dijelove gradiva. Model bi bio treniran na učitanim materijalima koji bi mu bili primarni izvor istine, a po potrebi mu se može omogućiti i pristup internetu radi proširenja opsega znanja. Ova implementacija zahtijevala bi precizno strukturiranje pravila odgovora kako bi točnost informacija bila na što višem nivou.

Literatura

- [1] Microsoft Learn, „Microsoft Learn - Interactive Documentation and Tutorials“. [Na internetu]. Dostupno na: <https://learn.microsoft.com/>
- [2] Udemy, „Udemy - Online Learning Platform“. [Na internetu]. Dostupno na: <https://www.udemy.com/>
- [3] Moodle, „Moodle - Open-source learning platform“. [Na internetu]. Dostupno na: <https://moodle.org/>
- [4] M. Jones, J. Bradley, i N. Sakimura, „RFC 7519: JSON Web Token (JWT)“. [Na internetu]. Dostupno na: <https://datatracker.ietf.org/doc/html/rfc7519>
- [5] Angular Team, „Angular Documentation and Developer Guides“. [Na internetu]. Dostupno na: <https://angular.dev/>
- [6] Tailwind Labs, „Tailwind CSS - Utility-first CSS framework“. [Na internetu]. Dostupno na: <https://tailwindcss.com/>
- [7] PrimeFaces, „PrimeNG - UI Component Library for Angular“. [Na internetu]. Dostupno na: <https://www.primefaces.org/primeng/>
- [8] W3C, „Web Content Accessibility Guidelines (WCAG) 2.2“. [Na internetu]. Dostupno na: <https://www.w3.org/TR/WCAG22/>
- [9] Microsoft, „.NET 10 Documentation“. 2026. [Na internetu]. Dostupno na: <https://learn.microsoft.com/dotnet>
- [10] Microsoft, „ASP.NET Core Documentation“. 2026. [Na internetu]. Dostupno na: <https://learn.microsoft.com/aspnet/core>
- [11] Microsoft, „Entity Framework Core Documentation“. 2026. [Na internetu]. Dostupno na: <https://learn.microsoft.com/ef>
- [12] S. Ramírez, „FastAPI - high performance, easy to learn, fast to code“. [Na internetu]. Dostupno na: <https://fastapi.tiangolo.com/>
- [13] PyMuPDF contributors, „PyMuPDF / pymupdf4llm - PDF and document text extraction“. [Na internetu]. Dostupno na: <https://pymupdf.readthedocs.io/>
- [14] python-docx authors, „python-docx - Read and write Microsoft Word .docx files“. [Na internetu]. Dostupno na: <https://python-docx.readthedocs.io/>

- [15] python-pptx authors, „python-pptx - Create and modify PowerPoint files“. [Na internetu]. Dostupno na: <https://python-pptx.readthedocs.io/>
- [16] PostgreSQL Global Development Group, „PostgreSQL Documentation“. 2024. [Na internetu]. Dostupno na: <https://www.postgresql.org/docs/>
- [17] Microsoft Azure, „Microsoft Azure Documentation“. 2026. [Na internetu]. Dostupno na: <https://learn.microsoft.com/azure>
- [18] Microsoft, „SPA Proxy in ASP.NET Core Documentation“. 2026. [Na internetu]. Dostupno na: <https://learn.microsoft.com/aspnet/core/client-side/spa/intro>
- [19] GitHub Actions Staff, „GitHub Actions Documentation“. [Na internetu]. Dostupno na: <https://docs.github.com/actions>
- [20] Marked contributors, „Marked - Markdown parser and compiler“. [Na internetu]. Dostupno na: <https://marked.js.org/>
- [21] PrismJS contributors, „PrismJS - Lightweight syntax highlighting“. [Na internetu]. Dostupno na: <https://prismjs.com/>
- [22] Pydantic authors, „Pydantic - Data validation using Python type hints“. [Na internetu]. Dostupno na: <https://pydantic.dev/docs/validation/latest/get-started/>
- [23] Google DeepMind, „Gemini API Documentation and Developer Guides“. 2026. [Na internetu]. Dostupno na: <https://ai.google.dev/gemini-api/docs>

Izjava o korištenju umjetne inteligencije

Osvrt na korištenje umjetne inteligencije: Umjetna inteligencija pri izradi ovog rada korištena je u sljedećim oblicima:

- Provjera gramatike i pravopisa - ChatGPT (<https://chatgpt.com/>)
- Pretraživač informacija i izvora - Gemini (<https://gemini.google.com/>)
- Učenje i savladavanje programskih jezika i radnih okvira - Claude (<https://claude.ai/>)

Izjava o korištenju umjetne inteligencije: Potvrđujem da sam tijekom izrade ovog završnog rada koristio umjetnu inteligenciju u skladu s Politikom primjerenog korištenja umjetne inteligencije na Fakultetu elektrotehnike i računarstva, te da umjetna inteligencija nije zamijenila moje kritičko razmišljanje niti moj doprinos.